

Structural Estimation in Corporate Finance: Project Plan

Zhaoxuan Wang

Version: June 21, 2026

Contents

1	Commercial Product	2
1.1	Product 1: Corporate Finance Advisory (CFA)	2
1.2	Product 2: Debt Capital Markets (DCM)	3
1.3	Product 3: Leveraged Finance and Private Credit	4
1.4	Model risk and deployment issues	6
2	Model and Estimation	8
2.1	Purpose, scope, and notation	8
2.2	Model	9
2.3	Estimation method overview	12
2.4	Block 1: value and policy networks	14
2.5	Block 2: grid refinement, simulation, and moments	18
2.6	Block 3: moment-surrogate networks and estimation	21
2.7	Minimum loss functions, adaptive shrinkage, and identification	23
2.8	Asynchronous execution architecture	24
2.9	Hyperparameters	24
2.10	Application inputs: external values, bounds, targets, reference estimates	26
2.11	Seeding and reproducibility	27
2.12	TensorFlow implementation notes	28
3	Validation and Tests	32
3.1	Benchmark solver	32
3.2	Oracle test 1: Monte Carlo recovery of parameters and moments	33
3.3	Oracle test 2: policy network against benchmark	34
3.4	Economic property tests	34
3.5	Regression tests	36
3.6	Value and policy accuracy against VFI	37
3.7	Monte Carlo recovery results	37
	Appendix: Interview QA	42

Chapter 1

Commercial Product

This chapter proposes a variety of commercial products developed from the project. Each section describes one product category: context, potential buyer, commercial value, example usage, and pricing strategy.

Across all product category, the core engine is a *quantitative model and system* consisting of:

- **Information input:** data, parameters, assumptions, theories, configurations;
- **Processing:** machine learning, statistical and econometric methods, numerical optimization;
- **Information output:** quantitative estimates and forecasts.

Importantly, this model does not directly make financial decisions or actions. It is used as a tool to support the user's judgment and decision-making.

1.1 Product 1: Corporate Finance Advisory (CFA)

Corporate Finance Advisory (CFA) is a global, multi-disciplinary solutions team specializing in structured M&A and capital markets that is focused on differentiating J.P. Morgan's Investment Banking services. CFA works with firms in all industries and partners across the bank to advise on solutions to problems with multifaceted, complex and unique features.¹

The model typically takes the corporate client's financial data and other information as input, and produce a credible interval of the optimal capital structure and policies as key outputs. These outputs are further processed to analyze counterfactual scenarios and generate other enhanced insights.

1.1.1 Buyer

Potential buyer is JPMC's CFA team and similar teams in other banks. This product directly supports:

- **Corporate finance solutions:** Provide analysis and recommendations on capital structure, capital allocation and shareholder distribution policy.
- **Ratings advisory:** Strategic and tactical advice on management of rating agency relationships and optimization of capital structures for desired credit rating objectives.
- **Structuring and solutions:** Structure products and alternative financing solutions relating to strategic M&A and capital markets.

¹<https://www.jpmorgan.com/investment-banking/corporate-finance-advisory>

- **Enhanced insight generation:** Leverage new technologies to improve both data analytics and visualization across all product and coverage groups.

1.1.2 Example

A company holds \$2 billion in surplus cash and wants to return it to shareholders through a buyback, funded partly with new debt. Its board asks one question: does the plan raise value without threatening the company's credit rating? JPMorgan's CFA team advises on the answer. The standard approach relies on peer benchmarks, rating-agency ratio scorecards, stress scenarios, and experienced judgment. These can estimate how the company's ratios map to a likely rating, but they do not value the choice itself, weighing the gain in firm value against the added default risk in a unified, model-based framework.

The model answers the question directly. From the company's own financials, it simulates the buyback and measures two effects together: the change in firm value and the change in default risk. It shows that the full \$2 billion buyback raises value only modestly and lifts the chance of a one-notch downgrade to roughly 30%. A smaller \$1.5 billion buyback captures most of the value while keeping downgrade risk low. The team brings the board a clear tradeoff, grounded in the company's own numbers rather than peer averages.

1.1.3 Value and Pricing

CFA work is firm-specific advice on capital structure, allocation, payout, and ratings. The model produces exactly that counterfactual analysis quantitatively. For a banker, this is a differentiated pitch tool: "our proprietary model, estimated on your fundamentals, says X." This is also a way to standardize work that is currently bespoke and analyst-heavy.

- **Ceiling:** set by how much the bank believes the model lifts mandate wins and fees.
- **Floor:** set by the analyst hours it displaces.

Realistic pricing strategy: platform subscription plus per-user license (seats), with the value carried in the sales narrative rather than a success fee.

Ideal pricing strategy: causal attribution that pins down the success fee per deal (e.g., "our product uplifts 10% of the advisory fee"). We need to deploy causal inference tools for credible evaluation. For example, start with a randomized experiment that gives the tool to different teams internally, collect data on outcomes, and estimate the average treatment effects.

Reason: It is hard to quantify the product uplift of wins and fees. CFA is often a relationship and credibility product that supports the broader IB relationship, so part of the product's value is indirect and long-term.

1.2 Product 2: Debt Capital Markets (DCM)

J.P. Morgan's Debt Capital Market team assists clients — including corporations, financial institutions, private equity and strategic investors — on a wide range of debt financing strategies.²

1.2.1 Buyer

Potential buyer is JPMC's DCM team. This product directly supports:

²<https://www.jpmorgan.com/investment-banking/debt-capital-markets>

- **Bond pricing advisory:** provide an independent fair-value anchor based on model estimates from the firm's fundamentals, which can be used to compare the market spread the bank derived from similar-rated bonds trade.
- **Liability management and structuring:** help the client decide whether to refinance, change the debt mix, or pick a tenor and seniority. Our model enables full dynamic counterfactual analysis across alternative capital structures.

1.2.2 Value

The bank captures this value indirectly. The client receives a better result than a peer-based rule would deliver, which strengthens the relationship and the bank's case to win the lead role, where several million dollars of underwriting fees are earned. Competing banks, working only from comparables, cannot produce the same fundamentals-based anchor. The tool does not replace the banker's judgment. It gives that judgment independent support.

1.2.3 Example

A company plans to raise \$1 billion by issuing a bond, and JPMorgan's DCM team must advise on the price. The standard method anchors the price to where comparable bonds trade, the issuer's own outstanding debt and similar-rated peers, plus a small new-issue concession. This reflects the market's current read and the issuer's rating, not an independent estimate built from the issuer's own fundamentals.

The model prices the bond from the issuer's fundamentals. It finds the company is stronger than its rating peers and supports a tighter spread. Carrying that independent anchor into the deal, the bank prices the bond about 15 basis points tighter than comparables alone would have set. On a \$1 billion ten-year bond, that saves the issuer roughly \$1.5 million a year, about \$15 million over its life.

1.2.4 Pricing

DCM decisions attach to basis points on large events (e.g., issuance, M&A). An independent fair-value anchor that helps price a deal a few bps better, or that strengthens the pitch to win the book-runner role, both has a clear and large per-deal value.

Example: A company wants to raise two billion dollars by issuing a 10-year bond. It hires banks to run the sale, and our tool is used by the bank on this deal. There are two places value can come from.

1. **Winning the mandate:** If the tool's independent fair-value analysis makes the bank's pitch more convincing, the bank is more likely to be chosen as bookrunner. Suppose the bank's fee on a deal this size is a few million dollars.
2. **Better pricing for the client:** Suppose the tool shows the bond can price 5 bps tighter than the comparable-bond view suggested. On two billion dollars, 5 bps is 0.05%, which is one million dollars a year that the borrower saves. This wins more future mandates and builds the bank's reputation.

Pricing strategy: a share of the value the bank can credibly attribute to the tool across a year of deals it touches, where that value is mostly extra mandate fees and proven cost savings.

1.3 Product 3: Leveraged Finance and Private Credit

The model can be used as a fundamentals-based underwriting and monitoring tool for leveraged finance and private credit.

Leveraged finance: J.P. Morgan is the partner of choice for Corporate & Financial Sponsors across the globe with unmatched structuring, marketing and execution capabilities in Leveraged Finance. J.P. Morgan platform is the undisputed market leader in arranging Loans and underwriting bonds that go to fund event-driven financing transactions including Capital Expenditures, Acquisitions, Leveraged Buyouts and Dividend Recapitalizations.

Investment grade finance: J.P. Morgan works directly with issuers, including corporations and financial institutions seeking debt financing. J.P. Morgan advises clients on capital structure strategy, from regular-way bond refinancing or a simple bank loan to multi-billion-dollar capital raises across asset classes. Colleagues partner across capital markets products and country borders to deliver successful structuring, marketing, and pricing

Private credit: J.P. Morgan increases direct lending commitment to 50 billion USD. This strategic move is designed to extend the firm's direct lending capabilities and provide tailored private credit solutions to meet the evolving needs of clients.³

1.3.1 Buyer

Internally it supports JPMorgan's Leveraged Finance group and its direct lending platform in CIB. Externally it is sellable to direct lenders, private credit funds, and leveraged buyout sponsors.

- **Underwriting and leverage sizing:** Give an independent, fundamentals-based estimate of how much debt a borrower can sustainably carry, and its default probability over the life of the loan, as a check on the deal team's comparable-based view before capital is committed.
- **Risk-based pricing:** Produce a fair credit spread for the proposed structure from the borrower's own fundamentals, flagging when a deal underpays the lender for the risk taken.
- **Scenario and covenant stress testing:** Run dynamic counterfactuals across leverage, tenor, and downturn cases, quantifying how default risk and covenant headroom (the leverage and coverage tests) move if earnings fall. This is timely, since 2026 is widely seen as the private credit market's first real stress test.
- **Portfolio monitoring and early warning:** Re-estimate the model each quarter across the existing loan book to flag borrowers whose default risk is rising before it surfaces in the financials, supporting risk governance and investor reporting.

1.3.2 Value

The larger value lies in the deals that our tool warns the fund to avoid. A single default on a loan this size can cost the fund \$80 million or more. Avoiding one such loss may outweigh many years of the subscription.

1.3.3 Example

A private credit fund must decide whether to lend \$200 million to a company that a private equity firm is buying. Its analysts compare the deal to how similar companies were financed and conclude the loan is safe and fairly priced. This peer comparison is the industry standard, and it has one structural weakness. No peer company is this borrower carrying this much new debt. The fund's existing tools, comparison tables and rating-style scorecards, all rest on peers and historical averages. None can model this specific borrower under this specific debt load.

³<https://www.jpmorgan.com/about-us/corporate-news/2025/jpmorgan-increases-direct-lending-commitment-to-50-billion>

Our structural model closes this gap. It uses data of the borrower’s own financials and estimates the probability that the company cannot service the debt. It then computes the interest rate that fairly compensates the fund for that risk. In this case, our model confirms the loan is safe but flags the proposed rate as 50 basis points too low. The fund secures the higher rate. On a \$200 million loan, that adds \$1 million of income per year, about \$5 million over a five-year term.

1.4 Model risk and deployment issues

The key issues for product deployment in real life are:

- Model validation, risk management and compliance.
- Both bankers and clients need to trust our model and estimates
- Ongoing monitoring after launch and performance evaluation.

The primary obstacle is **model risk management and compliance**: the product must address the related laws, rules, and supervisory guidance when deployed inside a global bank like JPMorgan. Specifically, the product needs to pass the requirements on model validation, risk management, AI-tool-use rules, and data protection policies.

The second issue is that **both bankers and clients need to trust our model and estimates**. It not only requires our model to be well tested and validated. The product should be tailored to the client’s need and incorporate firm-specific information to make the estimates useful. A lot of work is needed to understand what the real needs of the client are, and how our model can be developed into a specialized tool to meet the need. This requires deeper industry knowledge, professional guidance, and collaboration with clients.

The third issue is **ongoing monitoring and improvement** after launch. Model design and estimates should be updated with new data and customer feedback. We need to develop credible metrics and methods to evaluate the effectiveness of our tool, and quantify its impact.

The plan considers two main jurisdictions: (1) the United States, where JPMorgan is supervised as a US bank, and (2) the Hong Kong SAR, where the bank operates through locally regulated entities.

Scope of the product. For classification purposes, the end product is defined as a quantitative/statistical model that consumes input data and configurations, and outputs statistical estimates. There are several important boundaries:

- The model is not a black box and is fully explainable.
- The model is not a generative model or an agentic system.
- AI-tool-use is restricted to coding implementation and testing, supervised and reviewed by human.

Most of the current AI-related obligations center on generative AI and agentic systems. Our model is designed such that it is not generative or agentic, and it is largely within the standard quantitative/statistical model category.

1.4.1 Key principles of compliance

This section provides a high-level summary of the model risk governance and compliance guidelines in the United States and Hong Kong SAR.

- **A human stays responsible.** Final accountability rests with named people, not with the model. The model is decision support: a banker or officer reviews and signs off, and the model never makes a

decision automatically on its own. (US: SR 26-2 governance; HK: HKMA Principle 1, SFC General Principle 9.)

- **No black box.** The model must be explainable to the people who rely on it and to independent reviewers, clearly enough that they can question and challenge it. We have to be able to say, in explicit terms, why the model produced a given number. (US: “effective challenge”; HK: HKMA Principle 3, “no black-box excuse.”)
- **Reproducibility and record-keeping.** The model version, spec, parameters, inputs, and the output that informed any advice must be retained for the applicable period and reproducible on demand. Build versioning and an audit log into the product from the start. Baseline validation should emphasize reproducibility control.
- **Independent validation before it goes live.** The model must be tested and approved before it is used, and the builder cannot also be the validator. (US: SR 26-2 conceptual soundness and effective challenge; HK: HKMA Principle 5, SFC Internal Control Guidelines.)
- **Concrete and complete documentation.** The design, assumptions, model choices, data sources, and test results must be documented well enough for a third party to follow and reproduce the work. In practice, if it is not written down, it does not count as done. (US: SR 26-2 conceptual-soundness documentation; HK: across the AI principles and the conduct code.)
- **Out-of-sample testing.** The model must be checked against actual outcomes on data it was not built on (out-of-sample back-testing), not merely shown to fit its own training data. (US: SR 26-2 outcomes analysis; HK: SPM CA-G-3, validation “should not be limited to back-testing.”)
- **Monitoring after launch.** Performance must be monitored over time, and the model re-checked or re-estimated when markets or conditions change. (US: SR 26-2 ongoing monitoring; HK: HKMA AI principles.)
- **Be honest about limitations.** Known weaknesses must be disclosed to users, and use restricted where the model is weak. For us this means reporting ranges rather than false precision where parameters are only weakly identified. (US: SR 26-2 on use limitations; HK: same expectation.)
- **Input data quality.** The data feeding the model must be appropriate and of good quality, and the data choices documented. (US: SR 26-2 data selection; HK: HKMA Principle 4, data quality.)
- **If sold, give the buyer enough to validate it.** A buyer institution stays accountable for any model it uses, so we must ship a transparency package that lets its own reviewers understand and check the model without us handing over proprietary code. (US: SR 26-2, Section VII; HK: third-party provider principles.)

Chapter 2

Model and Estimation

This chapter describes the algorithm, methods, technical stack, and implementation details for solving and estimating structural corporate finance models. The methodological reference is Duarte and Fonseca (2026).

2.1 Purpose, scope, and notation

This document is a self-contained build specification for reproducing the DF26 estimation pipeline end to end. It restates the model, the algorithm, and every implementation choice in the paper’s own notation, and strips out literature review, motivation, and results discussion except where context is needed to implement a step correctly. DF26 is the single source of truth here. Where the paper is silent or internally inconsistent, the implementation decision is recorded in Section 2.12.1 (Resolved design decisions).

Target stack. The reference implementation in DF26 is written in JAX. This build must be written in **TensorFlow (TF) and TensorFlow Probability (TFP)**. NumPy is allowed only where it is the clearly better tool for a one-time setup computation (for example, fixed quadrature nodes), and even then the result must be converted to a `tf.constant` and consumed natively in TF. Implementation notes on the JAX-to-TF mapping are in Section 2.12. No code appears in this document; code is produced by a separate agent.

Notation. β is the **estimated parameter vector**. The firm’s discount factor is the fixed constant $1/(1 + r_f)$ and is written out explicitly wherever it appears; β is reserved for the parameter vector. (DF26 uses β for both.)

The canonical order of the estimated parameter vector, used throughout DF26’s estimation text, is

$$\beta = (\theta, \rho, \sigma, \delta, \gamma_1, \gamma_0, \chi, c_f).$$

Bind each parameter to its bounds and value by name, not by table position; the paper’s tables sometimes print a different order, which is cosmetic.

Primes. A prime (′) denotes a next-period quantity. Uppercase letters are levels; lowercase letters are the same quantity expressed per unit of capital. The link is $C = c \cdot k$, $B = b \cdot k$, $C' = c' \cdot k'$, $B' = b' \cdot k'$.

2.2 Model

Time is discrete. A single infinitely lived firm combines capital and labor to produce output under productivity shocks, pays fixed operating costs before production, and finances itself with current profits, one-period risky debt, external equity, and internal cash. Two frictions matter: default carries deadweight (recovery) costs, and equity issuance carries fees.

2.2.1 Parameters

Estimated parameters $\beta = (\theta, \rho, \sigma, \delta, \gamma_1, \gamma_0, \chi, c_f)$:

Symbol	Meaning
θ	Returns to scale, $\theta \in (0, 1)$
ρ	Autocorrelation of log productivity
σ	Standard deviation of the productivity innovation
δ	Depreciation rate
γ_1	Convex capital adjustment cost coefficient
γ_0	Fixed capital adjustment cost coefficient
χ	Default recovery rate
c_f	Fixed operating cost

External (fixed) parameters, set following Gao, Whited, and Zhang (2021):

Symbol	Value	Meaning
r_f	0.02	Risk-free rate; discount factor is $1/(1 + r_f)$
α	0.30	Capital share
τ	0.20	Corporate tax rate
λ_0	0.007	Fixed equity issuance cost
λ_1	0.054	Proportional equity issuance cost
r_c	0	Interest earned on cash; this makes $\iota_c = 1$

2.2.2 State variables and controls

States: (z, k, b) .

- z : productivity. It evolves as an AR(1) in logs (Eq. 2) and enters operating surplus linearly (Eq. 3). The z in Eq. 3 is the reduced-form productivity shifter that has absorbed the $1/\nu$ exponent from the static labor choice; the spec treats the single state z as that shifter throughout (AR(1), Tauchen grid, value function, surplus).
- k : capital.
- b : net debt, $b = b^{\text{gross}} - c$, where b^{gross} is gross debt per unit of capital and c is cash per unit of capital. Negative b means a net cash position.

Controls: (i, b', c') .

- i : investment **rate**, defined so that $k' = (1 + i - \delta)k$ (Eq. 10). Note this is the rate, not the level I . The level satisfies $I = i k = k' - (1 - \delta)k$.
- b' : next-period **gross** debt per unit of next-period capital k' .
- c' : next-period cash per unit of next-period capital k' .

The next-period net-debt state is therefore $b' - c'$, which is why the continuation value in the Bellman equation is evaluated at $(z', k', b' - c')$.

2.2.3 Technology and production

Output:

$$y = z \left(k^\alpha \ell^{1-\alpha} \right)^\theta, \quad \alpha \in (0, 1), \theta \in (0, 1). \quad (1)$$

Log productivity follows an AR(1):

$$\log z' = \rho \log z + \sigma \varepsilon', \quad \varepsilon' \sim \mathcal{N}(0, 1). \quad (2)$$

Labor is flexible and hired at competitive wage w . Substituting its closed-form optimum gives the reduced-form operating surplus:

$$\pi(z, k) = z A_\pi k^\xi - c_f, \quad \xi \equiv \frac{\alpha \theta}{\nu}, \quad \nu \equiv 1 - (1 - \alpha)\theta. \quad (3)$$

In partial equilibrium the wage is normalized to $w = 1$, so A_π is a constant in (α, θ) :

$$A_\pi = \nu \left((1 - \alpha)\theta \right)^{\frac{(1-\alpha)\theta}{\nu}}.$$

Labor and the wage do not enter the dynamic problem.

2.2.4 Investment and adjustment costs

Capital accumulates as

$$k' = (1 - \delta)k + I. \quad (4)$$

Investment incurs a convex cost $\frac{1}{2} \gamma_1 I^2 / k$ and a fixed cost $\gamma_0 k \mathbf{1}\{I > 0\}$ that is proportional to capital and is paid whenever investment is positive.

2.2.5 Debt, default, recovery, and bond pricing

The firm issues one-period debt at an endogenous price q that reflects default risk.

Liquidation recovery. A defaulting firm is liquidated, and creditors recover

$$R(k', c') = c' k' + \chi(1 - \delta)k', \quad (5)$$

its cash plus the salvage value of depreciated capital, where χ is the recovery rate.

Default rule. The firm defaults when (i) its continuation equity value is negative and (ii) its liquidation proceeds do not cover its outstanding debt. With continuation value $V(z', k', b' - c')$, recovery R (Eq. 5), and outstanding gross debt $b' k'$, the default indicator is

$$\mathbf{1}_{\text{def}} = \mathbf{1}\{V(z', k', b' - c') < 0\} \cdot \mathbf{1}\{R(k', c') < b'k'\}.$$

Condition (ii), $R < b'k'$ (equivalently $c' + \chi(1 - \delta) < b'$), depends only on the controls and parameters, not on z' . It is a deterministic gate: it does not constrain i , b' , or c' , and it keeps the recovery rate $R/(b'k')$ below 1 whenever default occurs. Condition (i) alone, the limited-liability walk-away, governs firm exit in the simulation (Section 2.5.2) and the moment filter (Section 2.5.3); (ii) only gates whether creditors take a loss.

Bond price (discounted expected repayment):

$$q(z, k', c', b') = \frac{1}{1 + r_f(1 - \tau)} \mathbb{E}_{z'|z} \left[(1 - \mathbf{1}_{\text{def}}) + \mathbf{1}_{\text{def}} \frac{R(k', c')}{b'k'} \right]. \quad (6)$$

The gate (ii) and the ratio $R/(b'k')$ are constant across z' , so with $g = \mathbf{1}\{R < b'k'\}$ and $P_{\text{def}} = \Pr(V(z', k', b' - c') < 0)$, Eq. 6 collapses to

$$q = \frac{1}{1 + r_f(1 - \tau)} \left[(1 - g P_{\text{def}}) + g P_{\text{def}} \frac{R(k', c')}{b'k'} \right].$$

When $R \geq b'k'$ ($g = 0$) the bond is risk-free; when $R < b'k'$ ($g = 1$) the recovery rate is below 1.

2.2.6 Dividends and equity issuance (two subperiods)

Cash flow is split into a preproduction and a postproduction stage.

Preproduction (roll over debt, pay fixed operating cost):

$$d_1 = q \cdot b' \cdot k' - b \cdot k - c_f. \quad (7)$$

If $d_1 > 0$, the excess carries forward. If $d_1 < 0$, the firm issues equity at cost $\lambda_0 k + \lambda_1 |d_1|$ (fixed plus proportional).

Postproduction (collect output net of investment and adjustment costs, plus any carried excess):

$$d_2 = z A_\pi k^\xi + \max\{d_1, 0\} - I - \frac{1}{2} \gamma_1 I^2 / k - c' k' \iota_c - \gamma_0 k \mathbf{1}\{I > 0\}, \quad (8)$$

with $\iota_c = 1/(1 + r_c r_f(1 - \tau))$. Since $r_c = 0$, $\iota_c = 1$. If $d_2 > 0$ the firm distributes; if $d_2 < 0$ it issues equity at the same fixed-plus-proportional cost.

Net payout to shareholders (issuance costs subtracted wherever issuance occurs):

$$D = d_2 + d_1 \cdot \mathbf{1}\{d_1 < 0\} - (\lambda_0 k + \lambda_1 |d_1|) \cdot \mathbf{1}\{d_1 < 0\} - (\lambda_0 k + \lambda_1 |d_2|) \cdot \mathbf{1}\{d_2 < 0\}. \quad (9)$$

Reading of the terms: d_2 already includes the rolled-over positive preproduction excess; the $d_1 \cdot \mathbf{1}\{d_1 < 0\}$ term restores the negative preproduction cash flow that the rollover left out; the last two terms are the issuance costs in each subperiod where the firm taps equity.

2.2.7 Bellman equation

Equity value solves

$$V(z, k, b) = \max_{i, b', c'} \left\{ D(z, k, b, i, b', c') + \frac{1}{1+r_f} \mathbb{E}_{z'|z} [\max\{V(z', k', b' - c'), 0\}] \right\}, \quad (10)$$

with $k' = (1 + i - \delta)k$. The $\max\{V, 0\}$ term is limited liability: shareholders walk away when equity value is negative. Eq. 6 (bond price) and Eq. 10 (value) are jointly determined, because q depends on V through default and V depends on q through the dividend. The algorithm breaks this circularity with a target network during training and with alternation during grid refinement (Sections 2.4.5 and 2.5.1).

2.3 Estimation method overview

The method solves the model once for all parameter values and then estimates with fast gradient-based optimization. It has three blocks plus an asynchronous controller.

- **Block 1 (Section 2.4): solve the model for all parameters at once.** Train a value network and three policy networks that take states and parameters as inputs, using policy iteration. After training they return the solution for any parameter vector instantly.
- **Block 2 (Section 2.5): simulate moments.** For each sampled parameter vector, refine the network solution on a grid, simulate a firm panel, and compute the targeted moments. This builds a dataset of (parameters, moments) pairs.
- **Block 3 (Section 2.6): estimate.** Train one differentiable “moment-surrogate” network per moment on that dataset, then estimate parameters by minimizing a weighted distance between data and model-implied moments using Levenberg-Marquardt with analytical Jacobians from the surrogate networks.
- **Adaptive controller (Section 2.7):** profile the objective one parameter at a time to get “minimum loss functions,” use them to shrink parameter bounds during training, and use them as a global identification diagnostic.

The map below shows how the blocks connect and loop.

End-to-end loop (the controller’s schedule):

1. Initialize parameter bounds (Table A1, Section 2.10) and the state-bound formulas.
2. Continuously (Block 1) run value/policy training epochs (500 gradient steps each) drawing parameters and states from the current bounds.
3. Concurrently (Block 2) draw parameter vectors, refine on a grid, simulate panels, compute moments, and append (parameter, moment) rows to a shared dataset.
4. At the end of every epoch: retrain the 110 moment-surrogate networks on the accumulated dataset, run the Levenberg-Marquardt estimation to get updated estimates, compute the minimum loss functions, and evaluate moment-network out-of-sample R^2 .
5. After a 200-epoch warm-up, at the end of every epoch attempt to shrink the parameter bounds using the minimum loss functions (subject to the safeguards in Section 2.7). Recompute all normalizations whenever bounds change.
6. Continue until convergence of the estimation loss.

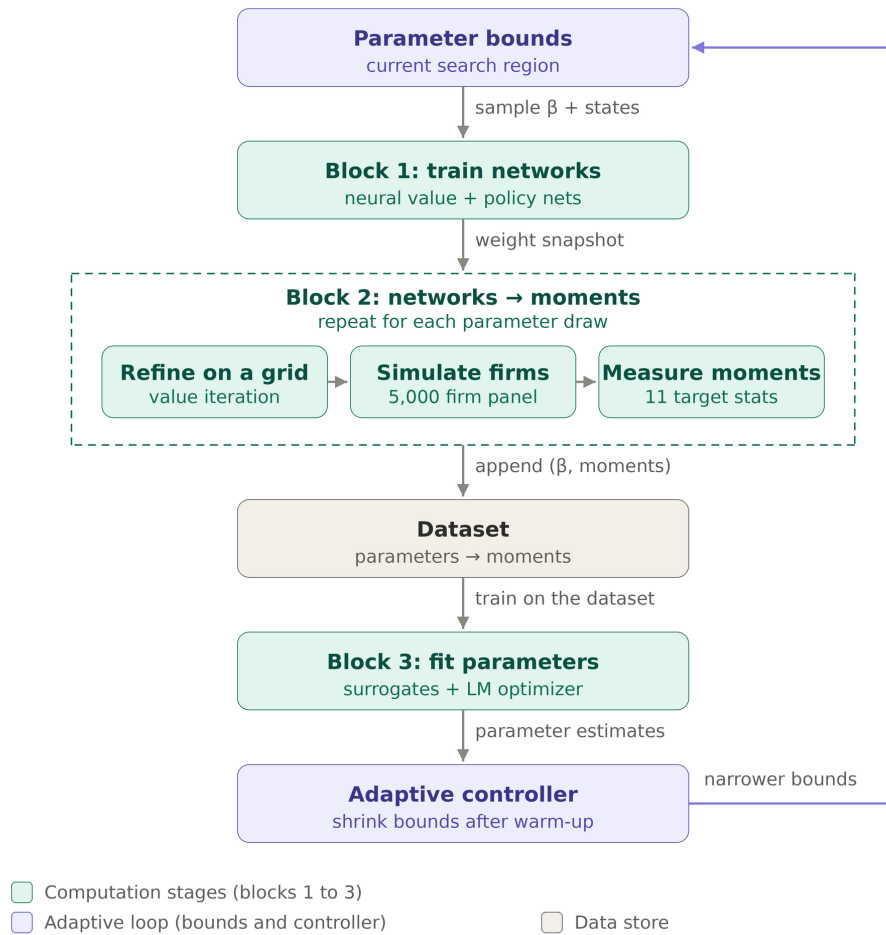


Figure 2.1: End-to-end estimation pipeline and adaptive loop

2.4 Block 1: value and policy networks

2.4.1 What the networks represent

- One **value network** approximates $V(z, k, b; \beta)$, returning a scalar.
- Three **policy networks**, one each for i, b', c' , each returning a scalar.

All four take the state vector and the parameter vector as inputs. They share the same architecture (Section 2.4.3). Each has its own weights.

2.4.2 Input normalization and bounds

Every input is mapped to $[-1, 1]$ by

$$\tilde{x} = \frac{2(x - x_{\min})}{x_{\max} - x_{\min}} - 1, \quad (24)$$

using that variable's $[x_{\min}, x_{\max}]$.

State bounds depend on parameters and are recomputed per parameter vector:

- Log productivity: $\log z \in \left[-m \frac{\sigma}{\sqrt{1-\rho^2}}, +m \frac{\sigma}{\sqrt{1-\rho^2}} \right]$ with $m = 2.5$. These are the unconditional ± 2.5 standard-deviation bounds of the AR(1). They depend on σ and ρ .
- Capital: $k \in [k_{ss}(z_{\min}), k_{ss}(z_{\max})]$. The frictionless investment Euler equation is

$$1 = \frac{1}{1+r_f} \mathbb{E}_{z'|z} \left[\xi z' A_{\pi}(k')^{\xi-1} + (1 - \delta) \right].$$

$k_{ss}(z)$ is its steady state for productivity held permanently at z , where $z' = z$ and the expectation collapses to z , giving the user-cost condition $\xi z A_{\pi} k^{\xi-1} = \delta + r_f$ and

$$k_{ss}(z) = \left[\xi z A_{\pi} / (\delta + r_f) \right]^{1/(1-\xi)},$$

evaluated at $z_{\min}$ and $z_{\max}$. This depends on $\delta, r_f, \theta, \alpha$ (through ξ and A_{π}) and ρ, σ (through $z_{\min}, z_{\max}$). A one-step target would instead use the conditional mean $\mathbb{E}_{z'|z}[z'] = z^{\rho} e^{\sigma^2/2}$ (the $Q = 5$ Gauss-Hermite rule of Eq. 30 evaluates it), but that bound is far tighter (it can cut the upper bound by about a third) and risks simulated capital leaving the grid, so the permanent- z form is used.

- Net debt: $b \in [-1, 2]$.

Because state bounds depend on parameters, the same raw state maps to different normalized inputs across parameter vectors.

Control bounds. Each control maps to its feasible interval through Eq. 26. The intervals are functions of the state and parameters only, not of each other, so the three sub-networks stay independent.

- Investment rate: $i \in [k_{\min}/k - (1 - \delta), k_{\max}/k - (1 - \delta)]$, a function of the current k . These are the rates for which next-period capital $k' = (1 + i - \delta)k$ stays in the capital box.
- Gross debt: $b' \in [0, 2]$. Gross debt is non-negative, and 2 is the net-debt ceiling.
- Cash: $c' \in [0, 1]$. Cash is non-negative. The upper bound is 1 because the next-period net-debt state $b' - c'$ must stay in its box $[-1, 2]$: with $b' \geq 0$, the deepest net cash position $b' - c' = -1$ is reached at $b' = 0$, which needs $c' \leq 1$. With $b' \in [0, 2]$ and $c' \in [0, 1]$, $b' - c'$ spans exactly $[-1, 2]$.

Parameter bounds are normalized with the same Eq. 24 using the current bounds. The initial bounds are in Table A1 (Section 2.10). They narrow over training (Section 2.7). Recompute the normalization whenever bounds change so inputs always sit in $[-1, 1]$.

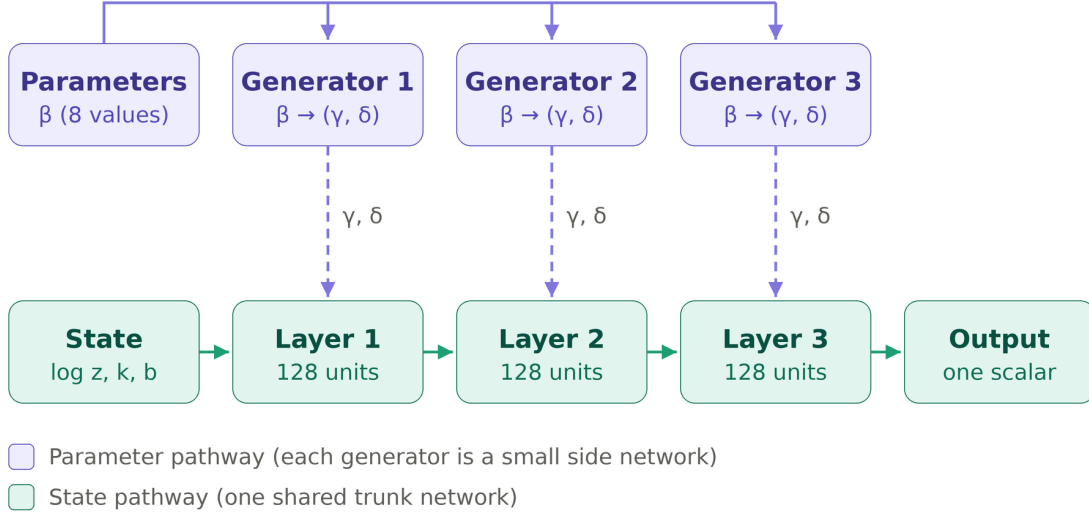


Figure 2.2: FiLM data flow in one value or policy network

2.4.3 Architecture: FiLM conditioning

States and parameters play different roles, so they enter differently. Feature-wise Linear Modulation (FiLM, Perez et al. 2018) keeps a compact trunk for the state-dependence and injects the parameter-dependence through a small generator that modulates each trunk layer, which avoids the large network a plain concatenation of all 11 inputs would need (DF26 Appendix A.12.1). The **state vector** ($\log z, k, b$) (normalized, 3-dimensional) enters the trunk; the **parameter vector** (normalized, 8-dimensional) enters only through FiLM.

One network is one trunk plus three generators, one generator per trunk hidden layer. The state flows only through the trunk; the parameters flow only through the generators, whose per-layer scale γ_l and shift δ_l rescale and shift that layer’s pre-activation (Eq. 25). Across the four networks (V, i, b', c') this is 4 trunks and 12 generators, trained together in one backward pass. The architecture stays small because the trunk learns the state shape once while the generators learn only the lower-dimensional map from parameters to per-layer gains and shifts, which gives a different effective function for each parameter vector.

For each trunk hidden layer l , a small auxiliary generator network maps the normalized parameter vector to a scale γ_l and a shift δ_l , each of the same width as the hidden layer. The layer output is

$$h_{l+1} = \phi(\gamma_l \odot (W_l h_l + c_l) + \delta_l), \quad (25)$$

where $\odot$ is element-wise multiplication, W_l, c_l are the layer’s weight matrix and bias, and ϕ is the activation.

Trunk. 3 hidden layers, 128 units each, SiLU activation $\phi(x) = x \sigma(x)$ where σ is the logistic sigmoid. The output layer is linear with no activation.

FiLM generator (one per trunk hidden layer, so 3 generators per network, with their own weights): input is the normalized parameter vector; 1 hidden layer of 32 units; output length 2×128 , split into γ_l (128)

and δ_l (128). The generator hidden-layer activation is SiLU, consistent with the trunk and with Table A2. Initialize the generator output layer so the modulation starts as the identity, $\gamma_l = 1$ and $\delta_l = 0$: set the output weights near zero, the γ_l bias to 1, and the δ_l bias to 0. Each FiLM layer is then a no-op at the start of training, so the trunk keeps its initial behavior and the generator learns departures from the identity as training proceeds.

Value network output. A single scalar interpreted directly as V (no squashing).

Policy network outputs. Each of the three control sub-networks outputs one raw scalar r , mapped to a feasible interval by

$$a = a_{\min} + (a_{\max} - a_{\min}) \cdot \sigma(r), \quad (26)$$

so a stays strictly inside $[a_{\min}, a_{\max}]$. For gross debt b' , subtract 4 from the raw output first: $b' = b'_{\min} + (b'_{\max} - b'_{\min}) \cdot \sigma(r - 4)$. Since $\sigma(-4) \approx 0.018$, this biases the initial debt policy low, which stabilizes early training before the bond-pricing function is learned. The feasibility bounds $[a_{\min}, a_{\max}]$ for each control are given in Section 2.4.2.

2.4.4 Training loop: policy iteration

Training alternates policy evaluation and policy improvement. One **epoch** is 500 gradient steps. Each step does the following in order.

Step 1 — sample. Draw a mini-batch of $N = 8,192$ state-parameter pairs. Sample parameter vectors uniformly from the current bounds. For each parameter vector, compute its parameter-dependent state bounds (Section 2.4.2) and sample $(\log z, k, b)$ uniformly within them.

Step 2 — policy evaluation. Compute the Bellman residual at each sampled point,

$$R_i = V(z_i, k_i, b_i) - D_i - \frac{1}{1+r_f} \mathbb{E}_{z'|z_i} [\max\{V(z', k'_i, b'_i - c'_i), 0\}], \quad (27)$$

with D_i the dividend at the current state under the current policy and $k'_i = (1 + i_i - \delta)k_i$. Update the value-network weights to reduce the mean squared residual,

$$L_V = \frac{1}{N} \sum_{i=1}^N R_i^2, \quad (28)$$

with one Adam step at learning rate 10^{-3} . The gradient of L_V flows through both the $V(z_i, k_i, b_i)$ term on the left and the continuation $\mathbb{E}[\max\{V, 0\}]$ term on the right; the policy networks are held fixed in this step.

Step 3 — policy improvement. Using the updated value network, update the three policy networks to maximize the Bellman right-hand side. The loss is its negative,

$$L_\pi = -\frac{1}{N} \sum_{i=1}^N \left(D_i + \frac{1}{1+r_f} \mathbb{E}_{z'|z_i} [\max\{V(z', k'_i, b'_i - c'_i), 0\}] \right), \quad (29)$$

with one Adam step at learning rate 10^{-3} . The gradient flows through the policy outputs (i_i, b'_i, c'_i) , which determine D_i and the next-period states, while the value-network weights are held fixed.

Then return to Step 1 with a fresh mini-batch. Learning rate decays multiplicatively by 1% per epoch, floored at 10^{-6} , for the value and policy networks (Section 2.9).

2.4.5 Bond-pricing circularity: target network

V and q are jointly determined. The circularity is broken with a **target network**: a copy of the value network whose weights are frozen for all gradient steps within an epoch and reset to the current value-network weights at the start of the next epoch. The target network is used **only** to compute the bond price q (through the default probability). The continuation value $\mathbb{E}[\max\{V, 0\}]$ in the Bellman residual uses the **current** value network. This makes q move slowly relative to V , giving V time to adjust before q is recomputed.

Default probability during training (analytic, no inner quadrature). Treat $V' = V(z', k', b' - c')$ as a function of the shock ε (with $z' = e^{\rho \log z + \sigma \varepsilon}$) using the **target network**. Take a second-order Taylor expansion of $V'(\varepsilon)$ around $\varepsilon = 0$:

$$V'(\varepsilon) \approx V'(0) + a \varepsilon + \frac{1}{2} c \varepsilon^2, \quad a = \left. \frac{dV'}{d\varepsilon} \right|_0, \quad c = \left. \frac{d^2 V'}{d\varepsilon^2} \right|_0,$$

with a and c from automatic differentiation. The default region in ε is the set where this quadratic is negative. Solve $\frac{1}{2} c \varepsilon^2 + a \varepsilon + V'(0) = 0$ (discriminant $a^2 - 2c V'(0)$) and take the region from the curvature: with $c > 0$ it is the open interval between the two real roots, or empty (so $P_{\text{def}} = 0$) when the roots are complex; with $c < 0$ it is the two tails outside the real roots, or all of $\mathbb{R}$ (so $P_{\text{def}} = 1$) when the roots are complex; with $c \approx 0$ treat V' as linear and use the half-line where $V'(0) + a\varepsilon < 0$. Integrate the standard normal density over that set with the standard normal CDF Φ to get $P_{\text{def}} = \Pr(V' < 0)$ in closed form. The bond price then multiplies this by the deterministic gate $g = 1\{R < b'k'\}$ (Section 2.2.5). Use this approximation only during network training; grid refinement (Section 2.5.1) computes q exactly.

Gradient management in policy improvement. In Eq 29 the bond price enters the dividend as a function of the controls, $q = q(k', b', c')$, through the recovery, the gate, and the default probability. Let the policy gradient flow through q with respect to the controls: the firm prices its own default risk, so riskier choices fetch a lower q , and the policy must internalize this exactly as the grid solver does when it prices each candidate control (Section 2.5.1); treating q as a detached constant would drop this price-impact channel and bias the policy toward over-issuing risky debt. The only thing detached here is the target network's weights, which are frozen for the epoch anyway; that freeze does not detach q 's dependence on the controls. Compute the Taylor coefficients a and c with nested forward-mode automatic differentiation (forward accumulators) rather than nested reverse-mode tapes, so this second derivative stays well-defined and stable when the outer policy gradient differentiates through it.

2.4.6 Conditional expectation: Gauss-Hermite quadrature

The conditional expectation in the Bellman equation is an integral against the standard normal:

$$\mathbb{E}_{z'|z}[f(z')] = \int_{-\infty}^{\infty} f(e^{\rho \log z + \sigma \varepsilon}) \frac{e^{-\varepsilon^2/2}}{\sqrt{2\pi}} d\varepsilon.$$

Gauss-Hermite quadrature with nodes $\{x_q\}$ and weights $\{w_q\}$ (defined for the weight e^{-x^2}) approximates it after the substitution $\varepsilon = \sqrt{2} x$:

$$\mathbb{E}_{z'|z}[f(z')] \approx \frac{1}{\sqrt{\pi}} \sum_{q=1}^Q w_q f(e^{\rho \log z + \sigma \sqrt{2} x_q}). \quad (30)$$

Use $Q = 5$ nodes (exact for polynomial integrands up to degree $2Q - 1 = 9$). This quadrature is the expectation used during network training. (Grid refinement and simulation use the discrete Tauchen chain instead; see Section 2.5.)

2.4.7 Smooth approximation of non-differentiable payoffs

The dividend (Eqs. 8 and 9) is non-differentiable in several places that block gradients during training: the three step indicators $1\{d_1 < 0\}$, $1\{I > 0\}$, $1\{d_2 < 0\}$; the kink $\max\{d_1, 0\}$ carried into d_2 ; and the term $d_1 \cdot 1\{d_1 < 0\}$ (a minimum) in D . During training only, replace each with a smooth version controlled by a temperature $\tau > 0$:

$$1\{x < 0\} \approx \sigma(-x/\tau), \quad (31)$$

$$\max(0, x) \approx \tau \log(1 + e^{x/\tau}), \quad (32)$$

$$\min(0, x) \approx -\tau \log(1 + e^{-x/\tau}). \quad (33)$$

Eq. 31 smooths the step indicators (for $1\{I > 0\}$ use the complement $\sigma(I/\tau)$); Eq. 32 smooths $\max\{d_1, 0\}$; Eq. 33 smooths $d_1 \cdot 1\{d_1 < 0\} = \min\{d_1, 0\}$; and the issuance-fee magnitudes $|d_1|, |d_2|$ follow as $\max\{x, 0\} - \min\{x, 0\}$, which the two softplus forms reproduce exactly. The Bellman continuation $\max\{V', 0\}$ is left in its exact form (its subgradient is enough for training), and the default indicator $1\{V' < 0\}$ is smoothed separately through the default probability P_{def} (Section 2.4.5), not here.

As $\tau \rightarrow 0$ each converges pointwise to the exact function; the smooth indicator transitions over a region of width about 4τ around $x = 0$, so the gradient is nonzero near the kink. Set $\tau = 10^{-3}$. Accuracy of the final solution is not very sensitive to τ because grid refinement uses the exact, non-smooth dividend.

2.5 Block 2: grid refinement, simulation, and moments

For each sampled parameter vector: refine the network solution on a grid (Section 2.5.1), simulate a panel (Section 2.5.2), and compute the moments (Section 2.5.3). Output one (parameter, moment) row.

2.5.1 Grid refinement

This step removes the dependence on the smooth approximations and makes the solution as accurate as value function iteration (VFI). It uses the exact, non-smooth dividend.

Grid sizes:

Dimension	Points
Productivity z (state)	11
Capital k (state)	15
Net debt b (state)	35
Next-period capital k' (control)	81
Gross debt b' (control)	91

Dimension	Points
Cash c' (control)	71

The grids span the state and control bounds in Section 2.4.2. The capital grids (k and the k' control grid) are log-spaced, with more points at low capital where the value function curves most. The debt and cash grids (b , b' , c') are uniform, since the net-debt range spans negative and positive values (so it cannot be log-spaced) and the gross-debt and cash ranges include zero. The 11 productivity points are the Tauchen nodes of the AR(1).

This spacing applies only to the Block 2 refinement grid. Network training (Section 2.4.4) instead samples states uniformly in $(\log z, k, b)$ to cover the whole state-parameter space evenly.

Procedure for one parameter vector:

1. **Initialize from the networks.** Evaluate V and the three policies on the full grid. This gives a grid-based value function and policy.
2. **Policy iteration**, each round consisting of:
 - **(a) Policy improvement.** At each state grid point, search for the control combination (k', b', c') that maximizes $D + \frac{1}{1+r_f} \mathbb{E}_{z'|z}[\max\{V', 0\}]$ using the exact dividend. Do not search the entire control grid. Build a candidate set per control dimension from: the current policy at the grid point, the policies at the immediate neighbors in each of the (z, k, b) dimensions, and small ± 1 grid-index perturbations around the current policy. This gives 9 candidate values per control dimension, hence $9^3 = 729$ combinations to evaluate at each grid point. The network starting point makes this local search sufficient.
 - **(b) Policy evaluation.** Given the improved policy, solve for V satisfying the Bellman equation under that fixed policy. Because of $\max\{V, 0\}$ this is a nonlinear fixed point. Solve it with a semismooth Newton method: at each iteration partition grid points by the sign of V (set $\max\{V, 0\} = V$ where $V > 0$ and 0 where $V \leq 0$); with the partition fixed the Bellman equation is linear in V and is solved with GMRES (generalized minimal residual). Recompute the partition from the new solution and repeat until it stabilizes.
3. **Recompute bond prices.** With the updated V , recompute $q(z, k', c', b')$ exactly on the full grid (Eq. 6). This uses the same two-condition default indicator as training, $\mathbf{1}\{V' < 0\} \cdot \mathbf{1}\{R < b'k'\}$ (Section 2.2.5); the only difference is that $\Pr(V' < 0)$ is evaluated exactly over the Tauchen nodes rather than by the training-time Taylor step.
4. **Repeat.** Return to step 2 with updated bond prices.

Run **six rounds** of steps 2 through 4. Expectations over z' on the grid use the 11-state Tauchen transition matrix. DF26 does not give the inner-solve tolerances, so use these tight defaults, matching the 10^{-10} that DF26 uses for the Levenberg-Marquardt estimation. Semismooth Newton (policy evaluation, step 2b): stop when the active-set partition is unchanged between two consecutive iterations, that is, when no grid point flips the sign of V ; as a safeguard, also stop when $\|V^{j+1} - V^j\|_\infty / (1 + \|V^j\|_\infty) < 10^{-10}$, and cap it at 50 iterations. GMRES (the linear solve inside each Newton step): relative residual $\|b - Ax\|_2 / \|b\|_2 \leq 10^{-10}$, restart every 50 Krylov vectors, at most 1000 matrix-vector products, warm-started from the current V . The six policy-iteration rounds are a fixed count, not a convergence loop.

2.5.2 Panel simulation

For each parameter vector, simulate $N_f = 5,000$ firms over $T = 300$ periods and discard the first $T_0 = 200$ as burn-in. Initialize each firm at $t = 0$ by drawing z from the Tauchen stationary distribution and (k, b)

from random grid points; the burn-in removes any dependence on this start.

- Productivity evolves on the 11-state Tauchen Markov chain built from Eq. 2. No interpolation is needed in z (look up the policy at the current z node).
- For (k, b) , which are continuous and generally off-grid, interpolate the grid-based policies with **bilinear interpolation** over the four nearest (k, b) grid points (weights proportional to proximity).
- Capital updates as $k' = (1 + i - \delta)k$. The next-period net-debt state is $b^{\text{gross}'} - c'$, where $(i, b^{\text{gross}'}, c')$ are the interpolated policy outputs.
- **Default and reseed.** When a firm's continuation value $V(z, k, b)$ falls below zero, the firm walks away (limited liability) and exits. Replace it by drawing a new (k, b) from random grid points while keeping the current z (productivity is exogenous and does not reset on default). This keeps the panel at N_f active firms and makes the cross-section reflect the ergodic distribution after burn-in.

2.5.3 Moment construction

After burn-in, form the panel as a sequence of consecutive period pairs. For each pair, denote current-period variables without primes and next-period variables with primes. Define:

$$\text{Investment rate: } i = (K' - (1 - \delta)K)/K, \quad (34)$$

$$\text{Operating income: } inc = (zA_\pi K^\xi - c_f)/(K + C), \quad (35)$$

$$\text{Debt ratio: } d = B^{\text{gross}'} / (K' + C'), \quad (36)$$

$$\text{Cash ratio: } cash = C' / (K' + C'), \quad (37)$$

$$\text{Cash saving: } \Delta c = (C' - C) / (K' + C'), \quad (38)$$

$$\text{Net debt: } net = B / (K + C). \quad (39)$$

All moments are computed over observations where the firm is not in default in either period of the pair, that is $V > 0$ and $V' > 0$.

The 11 targeted moments (in this exact order):

1. Mean investment rate.
2. Standard deviation of investment rate.
3. Mean operating income.
4. Standard deviation of operating income.
5. Autocorrelation of operating income: the OLS slope from regressing next-period inc' on current-period inc , both centered by their sample means over good observations.
6. Mean debt ratio.
7. Standard deviation of debt ratio.
8. Mean cash ratio.
9. Standard deviation of cash ratio.
10. OLS slope on net debt in the cash-saving regression: a multivariate OLS of cash saving Δc on net debt net and operating income inc , all three centered by their means.
11. OLS slope on operating income in that same cash-saving regression.

The debt, cash, and cash-saving ratios (Eqs. 36-38) use next-period quantities over next-period total assets $K' + C'$, following the appendix; the investment, operating-income, and net-debt measures (Eqs. 34, 35, 39) use current-period quantities, following the main text. The debt ratio uses gross debt, matching the data item DLC+DLTT scaled by total assets.

2.6 Block 3: moment-surrogate networks and estimation

2.6.1 Moment-surrogate networks

Train one network per moment ($M = 11$). Each is a feedforward network with **3 hidden layers of 32 units**, SiLU activation. Input is the normalized parameter vector (8-dim, normalized to $[-1, 1]$ with the current bounds). Output is one scalar (the predicted moment).

Learn $g_j(\beta) \approx \mathbb{E}[m_j | \beta]$ by minimizing the mean squared prediction error on the simulated dataset.

2.6.2 Cross-validation

Use $K = 10$ -fold cross-validation. Partition the dataset into 10 roughly equal folds. For each moment and each fold, train a network on the other 9 folds and hold out the remaining fold for validation. This produces $M \times K = 110$ networks, a per-moment out-of-sample R^2 (monitored during training), and 10 parameter estimates per moment whose spread measures sensitivity to the surrogate approximation.

2.6.3 Training the surrogate networks

Train all 110 networks simultaneously and continually: each epoch warm-starts from the current weights and runs **200 SGD passes** over the accumulated dataset (Adam, learning rate 10^{-4} , batch size 256), rather than re-initializing. Cap the dataset at 10,000 observations, keeping the 10,000 most recent. Store each row as the raw parameter vector with its moments, and normalize the parameters to $[-1, 1]$ with the current bounds at training time (Section 2.6.1), so a bound change is absorbed by renormalization and no stored normalization can go stale. The per-network loss is the mean squared prediction error over that network's training fold:

$$L_{j,k} = \frac{1}{|T_k|} \sum_{i \in T_k} (m_{i,j} - g_j(\beta_i))^2. \quad (40)$$

No learning-rate decay for the surrogate networks.

2.6.4 Estimation objective

For each fold k , minimize the weighted distance between data moments $\hat{m}$ and the fold's surrogate predictions $m_k(\beta)$:

$$\hat{\beta}_k = \arg \min_{\beta} (\hat{m} - m_k(\beta))' W (\hat{m} - m_k(\beta)), \quad (41)$$

where m_k stacks the 11 predictions from fold k and W is the weighting matrix (Section 2.6.7).

2.6.5 Levenberg-Marquardt with analytical Jacobian

Solve Eq. 41 with Levenberg-Marquardt (LM), using the Jacobian $J = \partial m_k / \partial \beta$ (an 11×8 matrix). The Jacobian is computed exactly by automatic differentiation through the surrogate networks. Convergence tolerances are 10^{-10} for function value, step size, and gradient norm, with at most 20 iterations per run.

2.6.6 Multiple restarts and bound enforcement

Run LM from **30 random starting points** per fold and keep the lowest-objective solution. Each start is drawn from a standard normal in the unconstrained space (below). With $K = 10$ folds and 30 restarts, the estimation step runs 300 LM optimizations.

Enforce bounds by reparametrizing each parameter through a sigmoid:

$$\beta^j = \underline{\beta}^j + (\bar{\beta}^j - \underline{\beta}^j) \cdot \sigma(x^j), \quad (42)$$

with $x^j \in \mathbb{R}$. LM searches over $x = (x^1, \dots, x^8)$. The Jacobian with respect to x follows from the chain rule, $\partial m / \partial x = (\partial m / \partial \beta)(\partial \beta / \partial x)$, with $\partial \beta^j / \partial x^j = (\bar{\beta}^j - \underline{\beta}^j) \sigma(x^j)(1 - \sigma(x^j))$. Note the surrogate input is itself the normalized parameter (Eq. 24), so the full Jacobian also carries the constant normalization derivative; let automatic differentiation handle the whole chain.

2.6.7 Weighting matrix

W is the inverse of the moment variance-covariance matrix, estimated from the data with influence functions following Erickson and Whited (2002), clustered at the firm level.

For observation i and moment j , the influence function ψ_{ij} captures observation i 's contribution to the sampling variability of moment j :

- **Means:** $\psi_{ij} = x_{ij} - \hat{\mu}_j$.
- **Standard deviations:** $\psi_{ij} = ((x_{ij} - \hat{\mu}_j)^2 - \hat{\sigma}_j^2) / (2\hat{\sigma}_j)$.
- **Autocorrelation:** $\psi_i = \tilde{x}_i \tilde{\varepsilon}_i / \hat{Q}$, where $\tilde{x}_i$ is the centered lagged profitability, $\tilde{\varepsilon}_i = \tilde{y}_i - \hat{\rho} \tilde{x}_i$ is the OLS residual, and $\hat{Q} = \frac{1}{N_p} \sum \tilde{x}_i^2$.
- **Regression coefficients:** $\psi_i = \left(\frac{1}{N_{\text{reg}}} X'X \right)^{-1} x_i \varepsilon_i$, where x_i is the 2×1 vector of centered regressors and ε_i is the OLS residual.

Observations that do not contribute to a moment (for example, those without a valid lag for the autocorrelation) get $\psi_{ij} = 0$.

Stack the influence functions into an $N \times 11$ matrix Ψ . To handle within-firm serial correlation, sum the influence functions across observations belonging to each firm g , giving a $G \times 11$ matrix of firm-level totals. The cluster-robust covariance is

$$\hat{\Sigma}_{jl} = \frac{1}{N_j N_l} \sum_{g=1}^G \left(\sum_{i \in g} \psi_{ij} \right) \left(\sum_{i \in g} \psi_{il} \right), \quad (43)$$

where N_j, N_l are the effective sample sizes for moments j, l (they differ because moments needing lags use fewer points). Then $W = \hat{\Sigma}^{-1}$.

In practice work with the Cholesky factor $W^{1/2}$ such that $W = (W^{1/2})' W^{1/2}$, and normalize it by its median column norm. This does not change the estimates but improves conditioning.

Reproducing W exactly requires the Compustat micro data (Section 2.10; see Section 2.12.1).

2.7 Minimum loss functions, adaptive shrinkage, and identification

2.7.1 Minimum loss functions

For each parameter β^j , the minimum loss function measures the best achievable fit when β^j is held at a value and all other parameters adjust optimally:

$$L(\beta^j) = \min_{\beta^{-j}} (\hat{m} - m(\beta^j, \beta^{-j}))' W (\hat{m} - m(\beta^j, \beta^{-j})), \quad (44)$$

where β^{-j} are all parameters other than β^j . Evaluate it at **31 evenly spaced values** of β^j across its current bounds. At each grid point run the full LM estimation (30 restarts, $K = 10$ folds) over β^{-j} and take the **median** across folds. Also record, at each grid point, the standard deviation across the 10 folds; report bands of ± 2 pointwise standard deviations whenever displaying these curves.

2.7.2 Adaptive bound shrinkage

Use the minimum loss functions to narrow bounds during training, concentrating computation near the estimates. Begin only after a 200-epoch warm-up. After that, attempt a shrink at the end of every epoch.

Per-parameter level rule. For each β^j , find β^{j*} that minimizes $L(\beta^j)$. Given a tolerance $\Delta > 0$, set the new interval to the contiguous region around β^{j*} where $L(\beta^j) \leq L(\beta^{j*}) + \Delta$. Apply this to each of the 10 folds' minimum loss functions, and for each parameter take the interval spanning the lowest and highest endpoints across folds.

Identification guard (prevents shrinking weakly identified parameters). Before applying the level rule to a parameter, compare $L(\beta^{j*})$ to the 90th-percentile loss. If the 90th-percentile loss is not at least **three standard deviations** above the minimum, do not shrink that parameter. The standard deviation is the median across grid points of the pointwise across-fold standard deviation; before computing it, recenter each fold's curve around its own minimum (level differences across folds do not affect which β^j minimizes the loss).

Containment guard (keeps plausible parameters). A candidate region is admissible only if it still contains both of: (1) among the 500 most recent sampled parameters, the 50 whose simulated moments are closest to the targets in the same weighted-GMM norm used to estimate β ; and (2) every LM estimate produced in the current estimation step (all 30 restarts and 10 folds). If the candidate region would exclude any vector in these two sets, it fails.

Choosing the tolerance via volume. Do not pick Δ directly; instead pick the **target volume fraction** v : the new region's volume (the product of interval widths over parameters that pass the identification guard) as a fraction of the pre-shrink volume. For each candidate v , find the Δ that gives that volume. Start at $v = 0.80$ (a 20% volume cut) and search for the smallest $v \in [0.05, 1.0]$ that satisfies the containment guard. If no v in that range satisfies it, leave the bounds unchanged this round. After shrinking, all subsequent operations use the narrower bounds. Recompute the parameter and state normalizations from the new bounds so inputs stay in $[-1, 1]$; do not re-initialize the value, policy, or surrogate networks, since continuous training and the per-epoch surrogate updates let them re-adapt to the renormalized inputs within a few epochs (one reason the shrink is gradual).

2.7.3 Global identification diagnostic

To separate weak identification from misspecification, compute the minimum loss functions using **simulated moments at the estimated parameters** as the target rather than the data moments. Simulate at $\hat{\beta}$ (Section 2.5 process) to get model-implied moments $\tilde{m}$, then compute

$$L(\beta^j) = \min_{\beta^{-j}} (\tilde{m} - g(\beta^j, \beta^{-j}))' W (\tilde{m} - g(\beta^j, \beta^{-j})), \quad (45)$$

over 31 grid points of β^j , with 30 LM restarts over β^{-j} and 10 folds. The diagnostic is twofold: check whether each $L(\beta^j)$ has a unique global minimum (a sharp minimum means identified; a flat curve means weakly identified), and check whether this second estimation recovers the parameter vector used to generate $\tilde{m}$.

2.8 Asynchronous execution architecture

This concurrency is the source of the method’s speed and is part of the design, not an optional optimization. Two kinds of process run at the same time on four GPUs and share the network weights.

- **GPU 1 (trainer).** Runs the Block 1 value/policy training loop continuously, cycling through epochs of 500 gradient steps. It holds the value- and policy-network weights in `tf.Variables` and updates them in place at every gradient step.
- **GPUs 2 to 4 (collectors).** Each one, at the start of each parameter batch, reads a snapshot of the trainer’s current weights, then draws parameter vectors from the current bounds, evaluates the snapshot networks on the grid, refines the grid solution (Section 2.5.1), simulates panels (Section 2.5.2), computes moments (Section 2.5.3), and appends (parameter, moment) rows to the shared dataset.

No GPU waits for any other. Because the trainer keeps improving the weights, each collector batch starts from a better solution than the last, so the dataset improves over time. The grid refinement (Section 2.5.1) corrects whatever the snapshot network gives, so the moments are accurate at every stage and only get cheaper to compute as training proceeds; the 10,000-observation cap (Section 2.6.3) keeps the surrogates trained on the most recent, best collections.

At the end of every epoch the controller retrain the moment-surrogate networks on the current dataset, evaluates their held-out R^2 , runs the Levenberg-Marquardt estimation for updated estimates, and computes the minimum loss functions. After the 200-epoch warm-up it also attempts a bound shrink, often declining because of the identification and containment guards (Section 2.7).

Do not serialize this into a train-then-collect alternation between Block 1 and Blocks 2 to 4: that removes the concurrency the method relies on for its speed. A single-process serial mode is acceptable only as a debugging aid on one GPU; it runs the same math far more slowly. See Section 2.12 for the TF snapshot mechanism.

2.9 Hyperparameters

Value and policy networks

Setting	Value
Hidden layers	3
Units per layer	128
Activation	SiLU
FiLM generator	1 layer, 32 units
Optimizer	Adam
Learning rate	10^{-3} , decay 1% per epoch, floor 10^{-6}
Mini-batch size	8,192
Gradient steps per epoch	500

Moment-surrogate networks

Setting	Value
Hidden layers	3
Units per layer	32
Activation	SiLU
Optimizer	Adam
Learning rate	10^{-4} (no decay)
Mini-batch size	256
SGD passes per update	200
Cross-validation folds K	10
Max training observations	10,000 (use most recent)

Quadrature and grid

Setting	Value
Gauss-Hermite nodes Q	5
State grid $n_z \times n_k \times n_b$	$11 \times 15 \times 35$
Control grid $n_{k'} \times n_{b'} \times n_{c'}$	$81 \times 91 \times 71$
Tauchen standard deviations m	2.5

Estimation

Setting	Value
Optimizer	Levenberg-Marquardt
Restarts per fold	30
Convergence tolerance	10^{-10} (function, step, gradient)
Max iterations per restart	20

Simulation

Setting	Value
Firms N_f	5,000
Periods T	300
Burn-in T_0	200

Minimum loss functions and shrinkage

Setting	Value
Grid points per parameter	31
Target volume fraction (initial)	0.80
Warm-up before shrinking	200 epochs
Identification guard threshold	90th-pct loss must exceed minimum by 3 SD
Containment set 1	50 closest of 500 most recent sampled parameters
Containment set 2	all 30 restarts $\times$ 10 folds of current LM estimates

Smoothing

Setting	Value
Temperature τ (Eqs. 31 to 33)	10^{-3}

2.10 Application inputs: external values, bounds, targets, reference estimates

External parameter values (Section 2.2.1): $r_f = 0.02$, $\alpha = 0.30$, $\tau = 0.20$, $\lambda_0 = 0.007$, $\lambda_1 = 0.054$, $r_c = 0$ (so $\iota_c = 1$). The wage is normalized to $w = 1$ (Section 2.2.3).

Initial parameter bounds (Table A1). These are the starting bounds; they narrow during training. Bound by name (not by the table’s print order).

Parameter	Lower	Upper
θ (returns to scale)	0.60	0.82
ρ (autocorrelation)	0.50	0.80
σ (productivity SD)	0.05	0.20
δ (depreciation)	0.05	0.20
γ_1 (convex adj. cost)	0.05	1.00
γ_0 (fixed adj. cost)	0.001	0.20
χ (recovery rate)	0.001	0.90
c_f (fixed operating cost)	0.001	0.25

Data construction (DF26 Section 3.2.1). Compustat annual, 1970 to 2019. Keep U.S.-headquartered firms. Exclude financials (SIC 6000 to 6999), regulated (SIC 4900 to 4999), and quasi-governmental (SIC 9000+). Drop observations with any missing variable, total assets below \$10 million, negative sales, or sales or asset growth above 200%. Require at least four consecutive annual observations per firm. Variable mapping: cash = CH, debt = DLC+DLTT, operating income = OIBDP, investment = CAPX, each scaled by total assets (AT). Winsorize all ratios at the 1st and 99th percentiles by year. Because model firms are ex-ante homogeneous, remove firm fixed effects from the standard-deviation and autocorrelation moments by demeaning at the firm level and adding back the sample average.

Data-moment targets $\hat{m}$ (Table 1, Panel B). These are the values DF26 used. The processing recipe is fully specified (this section and Section 2.7), so reproducing them exactly requires only the same Compustat micro data; where that data is unavailable, use these values directly as the targets. Order matches Section 2.5.3.

#	Moment	Data target
1	Mean investment rate	0.065
2	SD investment rate	0.045
3	Mean operating income	0.095
4	SD operating income	0.100
5	Serial correlation of operating income	0.495
6	Mean debt ratio	0.271
7	SD debt ratio	0.148
8	Mean cash ratio	0.096
9	SD cash ratio	0.080
10	Coefficient of cash on net debt	0.094
11	Coefficient of cash on income	0.039

Reference estimates (for validation only; not inputs). These let you check that the pipeline lands in the right region. Standard errors in parentheses.

From Table 1, Panel A: $\theta = 0.796 (0.002)$, $\rho = 0.597 (0.007)$, $\sigma = 0.187 (0.002)$, $\delta = 0.066 (0.000)$, $\gamma_1 = 0.945 (0.014)$, $\gamma_0 = 0.014 (0.000)$, $\chi = 0.003 (0.024)$, $c_f = 0.028 (0.001)$. The recovery rate χ is weakly identified.

2.11 Seeding and reproducibility

Goal: given identical inputs and specifications, every random draw in the pipeline is reproducible. The grid refinement (Section 2.5) is already deterministic. Randomness enters in five seedable places and two that no seed can control.

Seedable sources:

1. Weight initialization, for the value and policy networks, the twelve FiLM generators, and the 110 surrogate networks.
2. Training-state sampling: the uniform draws of states and parameters in each Block 1 batch, and the collector parameter draws in Block 2.
3. Simulation: the Tauchen chain transitions, the initial (z, k, b) , and the reseed after default.
4. Surrogate training: mini-batch shuffling and the ten cross-validation fold assignments.
5. LM restarts: the thirty standard-normal starting points per fold.

Not seedable:

6. Hardware nondeterminism: nondeterministic GPU kernels and reduction order, which differ run to run and across machines at the floating-point level.
7. Async timing: collectors read weight snapshots at varying wall-clock times, so the dataset composition depends on timing, not on any seed.

Design. Use stateless RNG everywhere (`tf.random.stateless_*`), never the stateful global generator. A stateless op is a pure function of an explicit seed and a shape, so its output is identical regardless of call order, parallelism, or device. That is what makes the draws reproducible in a batched, multi-GPU, async pipeline, where a stateful counter would shift the whole stream whenever ops are reordered or run on more threads.

Use one master seed (an integer) per run. Derive every draw’s seed from it by folding in a purpose label and the natural indices, with `tf.random.experimental.stateless_split` and `stateless_fold_in`, then

pass the derived seed to the stateless op. Key each substream by what indexes it:

- `init`: (master, "init", layer_id)
- `training samples`: (master, "train", epoch, step)
- `collector parameters`: (master, "collect", collector_id, batch)
- `simulation`: (master, "sim", firm_id, period), with separate labels for the initial state and the reseed
- `surrogate`: (master, "surrogate", epoch, step) for shuffling, (master, "folds") for the split
- `LM`: (master, "lm", fold, restart)

Each draw is then a pure function of (master, purpose, indices), so a rerun reproduces it, and distinct substreams are independent without reasoning about a shared counter. Keying the simulation by (firm, period) makes the panel identical whether firms are simulated serially, batched, or across devices.

Reproducibility, in two tiers. Sources 1 to 5 are pinned exactly by the keyed stateless seeds: given the same master seed, the same inputs, and the same specification, the draws are identical on any hardware. Sources 6 and 7 are not. The full async production run is therefore not bit-reproducible; the seeds fix its draws but not the float math or the dataset timing. Exact numerical reproducibility is reserved for the regression slices (Section 3.5), which run single-GPU and serial with op-level determinism on (`tf.config.experimental.enable_op_determinism()` and single-threaded intra-op), on a fixed machine with pinned library versions. That combination makes the float math repeatable too, so the golden comparison can use a tight tolerance.

Caveat. Keras initializers read the global stateful generator by default. For the deterministic training slice, either pass explicit seeds to the initializers or generate the weights with stateless ops and assign them, so initialization is reproducible as well.

2.12 TensorFlow implementation notes

DF26 is JAX. Below are the points where the TF/TFP port needs care. No code is given; these are design directions for the building agent.

Automatic differentiation. Use `tf.GradientTape` for all gradients. The Bellman residual loss (Eqs. 27 to 29) needs gradients of V with respect to its own weights through both the left term and the continuation term; a single tape covers this. The default-probability quadratic approximation (Section 2.4.5) needs first and second derivatives of V' with respect to the scalar shock ε ; use nested tapes (an inner tape for the first derivative, an outer tape for the second) or `tape.gradient` applied twice. The estimation Jacobian $\partial m / \partial x$ (Section 2.6.5) is best obtained with `GradientTape.jacobian` (or `batch_jacobian` to vectorize over folds and restarts) through the surrogate networks plus the normalization and bound-reparametrization chain.

Optimizers. Use `tf.keras.optimizers.Adam`. For the value/policy learning-rate schedule (multiply by 0.99 each epoch, floor 10^{-6}), drive the optimizer's learning rate with a custom schedule or update it at each epoch boundary; do not use a fixed decay-steps schedule that ignores the floor. Moment networks use a constant 10^{-4} .

Gauss-Hermite nodes. Computing the 5 fixed nodes and weights once at startup is the one place NumPy is clearly the right tool (`numpy.polynomial.hermite.hermgauss`). Convert the result to a `tf.constant` and use it natively in the quadrature sum (Eq. 30). TFP has no direct GH helper.

Standard normal CDF. Use `tfp.distributions.Normal(0, 1).cdf` for the default-probability integration in Section 2.4.5 and for the Tauchen transition matrix.

Tauchen discretization. Build the 11-state chain from each parameter vector’s (ρ, σ) with the $m = 2.5$ coverage. Because the chain differs per parameter vector during simulation, vectorize the construction over the batch of parameter vectors using TFP’s `Normal.cdf` and TF array ops. Avoid Python loops over the batch.

Bilinear interpolation on the (k, b) grid. The simulation interpolates the grid policies over (k, b) with bilinear interpolation. Since k is log-spaced and b is uniform, map k to $\log k$ so both axes are regular, then use `tfp.math.batch_interp_regular_nd_grid`; or implement gather-based bilinear interpolation directly.

Levenberg-Marquardt. TFP has BFGS, L-BFGS, and Nelder-Mead, but no LM. Implement LM natively in TF: build the damped normal equations $(J'WJ + \mu I)\Delta = J'Wr$ using the analytical Jacobian from `GradientTape`, solve the linear system with `tf.linalg.solve`, and run the standard trust-region damping update on μ . Vectorize across the 10 folds and 30 restarts (300 independent small problems) with batched tensors and `tf.linalg.solve` on a leading batch dim.

Grid-refinement linear solve (policy evaluation). The active-set partition turns the Bellman fixed point into a linear system $(I - \frac{1}{1+r_f}A)V = D$, where A encodes the policy-implied next-state interpolation and the Tauchen z' -transition. DF26 uses GMRES. The state grid has $11 \times 15 \times 35 = 5,775$ unknowns, so a dense `tf.linalg.solve` is feasible on a GPU and is the simplest faithful option. If a matrix-free iterative solver is preferred, a GMRES routine must be implemented (TF has `tf.linalg.experimental.conjugate_gradient` for symmetric positive definite systems only, which does not fit a nonsymmetric operator). Wrap the semismooth Newton active-set loop and the six refinement rounds as TF control flow (`tf.while_loop`), and vectorize over the batch of parameter vectors processed together.

Cross-validated ensemble of 110 networks. Train all moment networks together by carrying leading batch dimensions on the weight tensors (one set of `[110, ...]` weights) and using batched matrix multiplies, or by `tf.vectorized_map` over the ensemble. Keep them on one device since they are tiny.

FiLM layers. Default to computing the FiLM math explicitly (compute each generator’s (γ_l, δ_l) from β , then apply Eq. 25 in each trunk layer), with all weights held in a `tf.Module` per network so the trunk and its three generators are created once and `trainable_variables` collects them for the tape and optimizer. This keeps the flow unified and avoids hand-managing variables. A subclassed Keras layer also works but adds `build()` and call-timing complexity and does not cleanly hold the generator-to-trunk coupling, so it is the fallback, not the default. Either way, never create variables inside a `tf.function`; create them once in the module and wrap only the step. See Section 2.4.3 for the generator activation and initialization.

Pitfalls that fail silently (issue: fix):

- Identity init: start $\gamma_l = 1$, $\delta_l = 0$ (near-zero generator output weights, γ bias 1, δ bias 0). Skipping it distorts the trunk from step one and can diverge.
- Modulation placement: apply (γ_l, δ_l) to the pre-activation $W_l h_l + c_l$ before SiLU, exactly as Eq. 25. Applying it after the activation changes the model with no error.
- Per-sample shapes: γ_l, δ_l are $[N, 128]$ and the modulation is element-wise $[N, 128] \times [N, 128]$. Do not collapse β to one (γ, δ) for the whole batch, which would make every firm share the same modulation.
- Separate generators: keep 12 distinct generators (3 per network, 4 networks). Sharing generator weights across layers loses the depth-varying conditioning.
- Exploding or dead units: γ scales the pre-activation, so a large γ blows activations up and $\gamma \approx 0$ kills units. Identity init plus the standard learning rate controls this; watch for dead units.

Multi-GPU asynchrony. TF supports explicit device placement (`tf.device`) and `tf.distribute`. In the 1-trainer-plus-3-collectors pattern (Section 2.8), the trainer holds the value- and policy-network weights in

`tf.Variables` on GPU 1 and updates them in place; each collector reads a snapshot of those weights at the start of every parameter batch, for example with `tf.Variable.read_value()` into local constants (single process) or a periodic weight copy over a producer-consumer channel (separate processes). Collectors append rows to a shared, concurrency-safe dataset buffer that the controller reads each epoch.

Graph compilation. Wrap the hot paths (training step, grid refinement, simulation step) in `tf.function` for XLA-style compilation, mirroring JAX’s JIT. Watch for retracing: keep shapes static, pass the parameter bounds and grids as tensors (not Python scalars that change), and avoid data-dependent Python control flow inside compiled functions.

Random number handling. Use stateless `tf.random.stateless_*` with per-purpose keyed seeds, never the stateful global generator; the seed design is in Section 2.11. The TF analogue of the JAX key-reuse pitfall is reusing one stateless seed across vectorized draws, so give each firm, period, and parameter vector its own seed path.

2.12.1 Resolved design decisions

These are implementation decisions this spec makes where DF26 is silent, ambiguous, or internally inconsistent. DF26 does not specify them. Each is a standard, defensible choice, and each points to where it is applied in the body.

Grid spacing (Section 2.5.1). The capital grids are log-spaced, denser at low capital where the value function curves most; the net-debt, gross-debt, and cash grids are uniform; network training samples states uniformly. DF26 is silent on spacing, so this follows common practice.

Default rule in grid refinement (Section 2.2.5). The bond-price default indicator is $1\{V' < 0\} \cdot 1\{R < b'k'\}$ (both conditions); firm exit and the moment filter use $V' < 0$ alone. This holds in training, simulation, and grid refinement. Because χ is small, the second condition almost always holds for indebted firms, so the rule coincides with $V' < 0$ there and matches DF26’s algorithm.

Debt-ratio moment definition (Section 2.5.3). The debt ratio uses the appendix form, gross debt over next-period total assets $B^{\text{gross}}/(K' + C')$, which matches the data (the gross item DLC+DLTT over total assets); the main text’s net-debt version does not. The cash and cash-saving ratios follow the appendix as well; investment, operating income, and net debt follow the main text. Exact reproduction of the reported targets depends on the micro data (see the data-moments decision below).

Expectation rule by step (Section 2.5.1). Training integrates over continuous z' with 5-node Gauss-Hermite quadrature; grid refinement uses the 11-state Tauchen transition matrix; the simulation runs the grid-refined policy and so inherits the Tauchen chain. Each step uses the rule that matches its representation of z .

Data moments and weighting matrix (Sections 2.10 and 2.7). The data processing is fully specified, so the only residual is the raw Compustat panel itself (a data diff from vintage and revisions). Section 2.10 fixes the sample, filters, variable mapping, winsorization (1st and 99th percentiles by year), and firm fixed-effect removal; Section 2.7 fixes the weighting matrix (inverse of the influence-function covariance, clustered at the firm level). Given the same micro data these reproduce the targets and the weighting matrix; where the data is unavailable, use the reported targets in Section 2.10 directly.

FiLM generator details (Section 2.4.3). The generator hidden-layer activation is SiLU, consistent with the trunk and Table A2, and the scale and shift start at the identity ($\gamma_l = 1$, $\delta_l = 0$) so each FiLM layer is a no-op at initialization. Both are safe, standard defaults that DF26 does not state.

Convergence criteria inside grid refinement (Section 2.5.1). The semismooth Newton stops when the active-set partition stabilizes (safeguard: 10^{-10} relative value change, at most 50 iterations); GMRES uses relative residual 10^{-10} , restart 50, at most 1000 matrix-vector products, warm-started. DF26 does not specify these; the values match its 10^{-10} estimation tolerance and target VFI-level accuracy.

Chapter 3

Validation and Tests

The pipeline needs tests that check whether its outputs are correct, not only that it runs. Three categories cover this:

- Oracle (known-answer): run on a case with a known answer (a trusted alternative method such as VFI, or data simulated from known true parameters) and check the pipeline recovers it. Sections 3.2 and 3.3.
- Economic property: check properties that theory says must hold for any input, including parameter comparative statics. Section 3.4.
- Regression: freeze a validated run under a fixed seed and pinned versions and check that later runs still match. This catches drift, not correctness. Section 3.5.

Section 3.1 builds the shared VFI benchmark the oracle tests rely on.

3.1 Benchmark solver

Both oracle tests need an independent, trusted solution of the model at a fixed parameter vector. Use value function iteration (VFI) on the discrete grid, built only for validation and never used in the estimation loop.

- Input: one fixed parameter vector β .
- Grids and operators: the state grid ($11 \times 15 \times 35$ over $(\log z, k, b)$) and control grid ($81 \times 91 \times 71$ over (k', b', c')) of Section 2.5.1, the 11-state Tauchen expectation, the exact (non-smoothed) dividend (Eqs. 7 to 9), and the joint value/bond-price fixed point (recompute q from the current V each sweep, as in Section 2.5.1 step 3).
- Solve: iterate the Bellman operator from a cold start (set V to the no-default, no-adjustment value, or to 0). At each state node, maximize the right-hand side by a global search over the full control grid. This differs from the grid refinement in Section 2.5.1, which does a local search around the network policy. Stop when the sup-norm change in V is below 10^{-8} and the argmax policy is unchanged between sweeps. Reuse the Section 2.5.1 policy-evaluation linear solve for the inner step; only the policy-improvement search is global.
- Output: VFI value and policy arrays on the grid. Off-grid values use the same log- k bilinear interpolation as Section 2.5.2.

VFI is intentionally exact and slow. It is the ground truth the network outputs are checked against.

3.2 Oracle test 1: Monte Carlo recovery of parameters and moments

This is the critical end-to-end test. It reproduces DF26 Section 4.1 and its Figures 1 and 2. The idea: build data from known true parameters with the trusted VFI solver, then check that the full pipeline recovers both the moments and the parameters.

Each draw is an independent recovery trial: run the full pipeline (Blocks 1 to 3 and the controller, including adaptive shrinkage toward that draw’s target) end to end per draw. The networks, the collection, and the surrogates depend only on the bounds, not on the target moments, so an efficient implementation may instead train them once over the fixed Table A1 bounds and repeat only the target-dependent Levenberg-Marquardt estimation per draw; this skips per-target shrinkage, so use it only if Figure V1 confirms the full-box surrogate is accurate (R^2 near 1).

Procedure:

1. Draw a true parameter vector β^* uniformly over the Table A1 bounds (Section 2.10). Fix the seed.
2. Solve the model at β^* with the VFI benchmark (Section 3.1).
3. Simulate a firm panel from the VFI policy using the Section 2.5.2 settings (5,000 firms, 300 periods, 200 burn-in, Tauchen chain).
4. Filter: if the panel has no defaulting firms, discard the draw and redraw. The recovery rate χ is not identified without defaults, so DF26 excludes these cases. Keep drawing until you have N kept draws. DF26 used $N = 40$; use 40 as the baseline and more for tighter coverage.
5. Compute the 11 targeted moments (Section 2.5.3) from the panel. These are the true targets $m^*(\beta^*)$.
6. Run the Levenberg-Marquardt estimation (Sections 2.6.4 to 2.6.6) against $m^*(\beta^*)$, giving the estimate $\hat{\beta}$.
7. Refine and simulate at $\hat{\beta}$ (Block 2, Sections 2.5.1 to 2.5.3) to get the fitted moments $\hat{m} = m(\hat{\beta})$.

Repeat steps 1 to 7 for all N kept draws, then produce two figures.

Figure V1 (true versus fitted moments, reproduces DF26 Figure 1). Eleven panels, one per moment in the Section 2.5.3 order. In each panel the x-axis is the true target moment m_j^* across draws and the y-axis is the fitted moment $\hat{m}_j$; draw each draw as a marker with a same-colour capped vertical spike for its 95% confidence interval (1.96 times the moment’s firm-clustered standard error, Section 2.6.7), over a dashed 45-degree line. Fix both axes to a common range with a square box, so the 45-degree line is exactly diagonal and coverage is read off the full range, not an auto-zoom. Title each panel with the moment name and annotate its R^2 , the standard linear-regression coefficient of determination (the squared correlation between true and fitted, in $[0, 1]$). Pass if $R_j^2 \geq 0.99$ for all 11 moments (DF26 reports 1.00).

Figure V2 (true versus fitted parameters, reproduces DF26 Figure 2). Eight panels, one per parameter ($\theta, \rho, \sigma, \delta, \gamma_1, \gamma_0, \chi, c_f$). In each panel the x-axis is the true β_j^* and the y-axis is the estimate $\hat{\beta}_j$; draw each estimate as a marker with a same-colour capped vertical spike for its 95% confidence interval (1.96 times the across-fold standard deviation of the estimate), over a dashed 45-degree line. Fix both axes to the parameter’s Table A1 range with a square box, so the 45-degree line is exactly diagonal and coverage and bias are visible across the full range. Annotate each panel’s R^2 (the squared correlation between true and fitted, in $[0, 1]$, same definition as Figure V1), and also report the per-parameter bias (mean of $\hat{\beta}_j - \beta_j^*$) and RMSE, which capture the offset and scale that R^2 alone does not. Pass if $R_j^2 \geq 0.95$ for at least seven of the eight parameters (DF26 reports ≥ 0.97 for seven of eight). χ is the expected weak one; a low R^2 there signals weak identification, not a bug.

Repeat the whole exercise under several seeds and report the spread of R^2 across seeds, since the draws and the simulation noise vary. A correctly specified data-generating process should give near perfect recovery;

a clear miss flags a pipeline bug rather than a modeling limit.

3.3 Oracle test 2: policy network against benchmark

This checks that the network solution matches the trusted on-grid solution at a fixed parameter vector. It reproduces and extends DF26 Appendix Figure A1.

Fix β at the reference estimates (Section 2.10), or any interior vector; state which. Build three solutions at that β : the raw Block 1 network policy, the refined network policy (Block 2 grid refinement at β , Section 2.5.1), and the VFI benchmark policy (Section 3.1).

Reference holding point for one-dimensional slices: median productivity $z = 1$, near-zero net debt $b = -0.03$, a fixed low capital k_{ref} (state the value, for example $k_{\text{ref}} = 0.2$), and the held parameters at the reference β .

Figure V3 (policy slices). A grid of one-dimensional slices. Rows are the policies i , b' , and c' (add V if useful). Columns are the 11 arguments: the three states k , b , z and the eight parameters θ , ρ , σ , δ , γ_1 , γ_0 , χ , c_f . For each (policy, argument) panel, vary that one argument over its range (state ranges from Section 2.4.2; parameter ranges from Table A1) on about 50 points, hold every other state and parameter at the reference point, and plot the policy.

- State-argument panels (columns k , b , z): produce one figure per state axis, each a 1x4 panel over V , i , b' , c' , so the (policy, state) combinations are complete. Overlay three curves: raw VFI as a thick dashed grey line, the Block 1 network in blue, and the refined network in bold red. Keep the refined curve the most visible, since it is the policy actually used.
- Parameter-argument panels (the eight parameters): produce one 1x3 figure per parameter (columns i , b' , c'), saved separately per parameter. Overlay two curves only, the Block 1 network (blue) and the refined network (red); raw VFI is omitted, because a single VFI solve fixes β and so cannot trace a parameter axis (see the optional extension).

The k column for rows i , b' , c' at $z = 1$ and $b = -0.03$ is exactly DF26 Appendix Figure A1. Reproduce it as the headline panel set, with the same three-curve legend.

Quantitative pass criteria (beyond the visual overlay). Over the state grid in the good region ($V > 0$), compare the refined network policy to VFI: report the maximum and mean absolute deviation per policy. The refinement targets VFI-level accuracy (Section 2.5.1, stop at 10^{-10}), so the refined policy should sit on the VFI curve; pass if the maximum relative deviation is below 1 percent per policy (or an absolute threshold scaled to each policy's range, stated in the output). Report the raw Block 1 deviation as well for context, with a looser sanity bound (for example within about 10 percent); the raw network is the unrefined approximation and is not required to match VFI tightly.

Optional comprehensive extension (skip at the current stage). To benchmark the parameter-axis panels against VFI, re-solve VFI at each point along a parameter axis (one VFI solve per grid point of that parameter, with the others held), producing a VFI curve for the parameter slices. This is expensive, one full VFI solve per point, but it validates the network's parameter dependence against the trusted method. Add it once the state-axis checks pass.

3.4 Economic property tests

These check properties that must hold for any input, not just on a known-answer case. Groups 1 to 5 are always-true properties; Group 6 is parameter comparative statics. Each check is hard or a diagnostic: a

hard check gates the suite (a violation fails the run); a diagnostic is computed and reported but never fails it. Hard checks are reserved for deterministic or theory-certain properties; anything expected but not guaranteed is a diagnostic. Shape and slice checks reuse the one-dimensional slice machinery and reference point of Section 3.3, evaluated on the VFI and refined-network solutions unless noted.

Group 1, correct bounds (all hard; deterministic domain, checked on every evaluation).

- Bond price $q \in [0, 1/(1 + r_f(1 - \tau))]$; the bracket is a repayment fraction in $[\text{recovery}, 1]$.
- Default probability $P_{\text{def}} \in [0, 1]$; recovery fraction $R/(b'k') \in [0, 1]$ when the gate $g = 1$.
- Controls land in their boxes: i keeps $k' = (1 + i - \delta)k \in [k_{\min}, k_{\max}]$; $b' \in [0, 2]$; $c' \in [0, 1]$; net debt $b' - c' \in [-1, 2]$. Checking the realized values catches a broken Eq. 26 mapping.
- Tauchen transition matrix: rows non-negative and summing to 1.
- Simulated states stay in the grid box; a leak signals an interpolation or bound bug.

Group 2, correct economics (monotone shape versus states).

Hard assertions (three): V increasing in z ; V decreasing in b ; i decreasing in k . Test: for each, draw about 20 hold configurations (the two held states and β sampled uniformly in the current bounds), sweep the varying state over its grid range on about 50 points, evaluate the VFI and refined solutions, keep interior points with $V > 0$, drop the two endpoints, take consecutive differences, ignore steps with $|\Delta| < \varepsilon_{\text{mono}} = 10^{-3} \times (\text{slice max} - \text{slice min})$, and require every remaining step to carry the asserted sign. Pass means zero violations across all configurations on both VFI and the refined network. The raw Block 1 network is reported, not gated.

Diagnostics (reported, not gated): V versus k (expected up at low k , may flatten or reverse at high k , since net debt is per unit of capital so the debt level scales as $b k$ under decreasing returns); i versus z (expected up); i versus b (expected down).

Skipped (no sign asserted): b' and c' versus every state are ambiguous and hump-shaped (Appendix A1b, A1c); validate their shape through the Section 3.3 overlays instead.

Group 3, correct mechanics (optimality and self-consistency).

Hard checks: the Bellman residual $R = V - D - (1/(1 + r_f)) E[\max\{V', 0\}]$ (Eq. 27) is small on the refined solution (tight, since refinement converges); the bond-price residual $|q - q(V)|$ (Eq. 6 at the solved V) is small on the refined solution; the accounting identities hold exactly when recomputed and compared ($k' = (1 + i - \delta)k$, next net debt = $b' - c'$, D from Eqs. 7 to 9 equals the D used, R equals Eq. 5); the weighting matrix $\hat{\Sigma}$ is symmetric positive definite (Cholesky succeeds); and no value, policy, price, moment, Jacobian, or weighting entry is NaN or infinite.

Diagnostics: the Bellman and bond residuals on the trained network (looser, since it is approximate); the Levenberg-Marquardt first-order condition (gradient near zero at $\hat{\beta}$, and the objective below nearby perturbations, which reads as flat for weakly identified directions like χ); and the policy-improvement objective (Eq. 29 right-hand side) having plateaued.

Group 4, corner cases (the deterministic checks above, evaluated at named extremes).

Hard checks: in the no-default region (low leverage or high χ , so the gate $g = 0$), q equals the risk-free price $1/(1 + r_f(1 - \tau))$ exactly, and a simulation with zero defaults runs to finite output without error; under high leverage ($R < b'k'$, $g = 1$), q stays in $(0, \text{risk-free})$ with recovery below 1; at every Table A1 corner the production constants are finite ($v = 1 - (1 - \alpha)\theta > 0$, so ξ and A_π finite), $k_{\min} > 0$ so per-capital terms are safe, and the Tauchen grid still covers the high- σ , high- ρ case.

Diagnostics: when the optimal control sits at a bound ($c' \rightarrow 0$, $b' \rightarrow 0$, or i at its limit), the sigmoid policy saturates near the edge while the grid refinement reaches the true edge, so report their agreement; and in the no-default case, report the χ identification flag (Section 2.7.3) rather than gating on it.

Group 5, correct consistency (two independent routes agree).

Hard check: surrogate moments versus simulate-then-measure at the same β , where the out-of-sample R^2 (Section 2.6.2) is at or above its acceptance threshold; a low R^2 invalidates the estimation.

Diagnostics: the training Taylor P_{def} versus the exact grid P_{def} (report the gap, expected small in the interior and largest near the default boundary, consistent with the paper treating the grid value as the accurate one); the Gauss-Hermite continuation versus the Tauchen continuation at the same state; and, optionally, normalization invariance (the network output at a fixed raw (z, k, b, β) is near-invariant to the active bounds on their overlap), which catches renormalization bugs.

Group 6, correct comparative statics (parameter monotonicities; all hard, no diagnostics).

Assert, holding the other seven parameters and the reference state fixed:

- V decreasing in c_f , in δ , in γ_1 , and in γ_0 .
- V increasing in χ .
- q non-decreasing in χ at fixed state, choices, and P_{def} .

Test: evaluate the Block 1 value network's V at the Section 3.3 reference state across about 20 hold configurations (the held state and the other seven parameters sampled uniformly in the current bounds), sweep the one parameter over its Table A1 range on about 50 points, take consecutive differences, ignore steps with $|\Delta| < \epsilon_{\text{mono}} = 10^{-3} \times (\text{sweep max} - \text{sweep min})$, and require every remaining step to carry the asserted sign. For q , evaluate Eq. 6 along the χ axis at fixed choices and P_{def} .

These are the only certain parameter signs. The value is a maximum, so it inherits any parameter shift that moves the payoff one way for every policy (a dominance argument needing no closed form), and the tax enters only through the bond-price debt term, so no profit-tax or depreciation shield offsets the cost and depreciation signs; q is a direct function of recovery $R = c'k' + \chi(1 - \delta)k'$. Control and moment signs are not asserted, because the controls are the maximizer, with competing channels and non-convex frictions that leave them ambiguous, and the moments add the stationary distribution; those are covered by the oracle recovery test (Section 3.2) and the Section 3.3 overlays. The value network evaluates any β cheaply, so a violation flags a bug; VFI along a parameter axis can confirm a few points but is not required.

3.5 Regression tests

This is a thin drift guard, not a correctness test; correctness is covered by the oracle and economic property tests above. It does not re-run training, which is slow and, under the async design, not reproducible (Section 2.11). Instead it freezes one validated run and re-checks only the deterministic, fast pieces.

Freeze once, from a run that passed the oracle and economic property tests, and commit as small files: the trained network checkpoint (value, three policies, twelve generators) and the surrogate checkpoint; a handful of fixed evaluation inputs (z, k, b, β) ; one parameter vector β_0 with its target moments; and the master seed.

The test reloads these and recomputes, comparing each output to its frozen golden:

- the network forward pass at the fixed inputs (V and the three controls), guarding the network, FiLM, and normalization code;

- the bond price q and dividend D at the fixed inputs via Eqs. 6 to 9, guarding the economic primitives;
- one on-grid refinement sweep from the frozen (V, q, policy) , guarding the Tauchen expectation, the semismooth-Newton step, and GMRES;
- a short fixed-seed simulation from the frozen policy (a few hundred firms, short horizon) and its moments, guarding the simulation and moment code;
- one LM solve on the frozen surrogate against β_0 , guarding the Jacobian, the weighting matrix, and the optimizer.

Each piece is a forward evaluation or a single operator application, so the whole test runs in seconds. Run it on a fixed machine with pinned versions, single-GPU and serial, with op-level determinism on (Section 2.11), so the float math is repeatable. Compare floats at $\text{rtol } 10^{-5}$, $\text{atol } 10^{-7}$; integer and structural outputs, such as default counts and the active-set partition, must match exactly.

When a change or a version bump alters the goldens on purpose, regenerate them under the same seed and commit them with a one-line note. The test then flags only unintended drift.

Optional: to also guard the training loop, add a short deterministic training slice, a handful of gradient steps from a fixed init and seed in the serial single-GPU mode, and freeze the loss trajectory.

3.6 Value and policy accuracy against VFI

Figure 3.1 reports the Block 1 policy-recovery test (Section 3.3): after grid refinement, the network value and policy functions reproduce the VFI benchmark across the state space.

3.7 Monte Carlo recovery results

Figures 3.2 and 3.3, with Tables 3.1 and 3.2, report the Monte Carlo recovery test (Section 3.2): data are simulated from known parameters, re-estimated through the full pipeline, and compared to the truth.

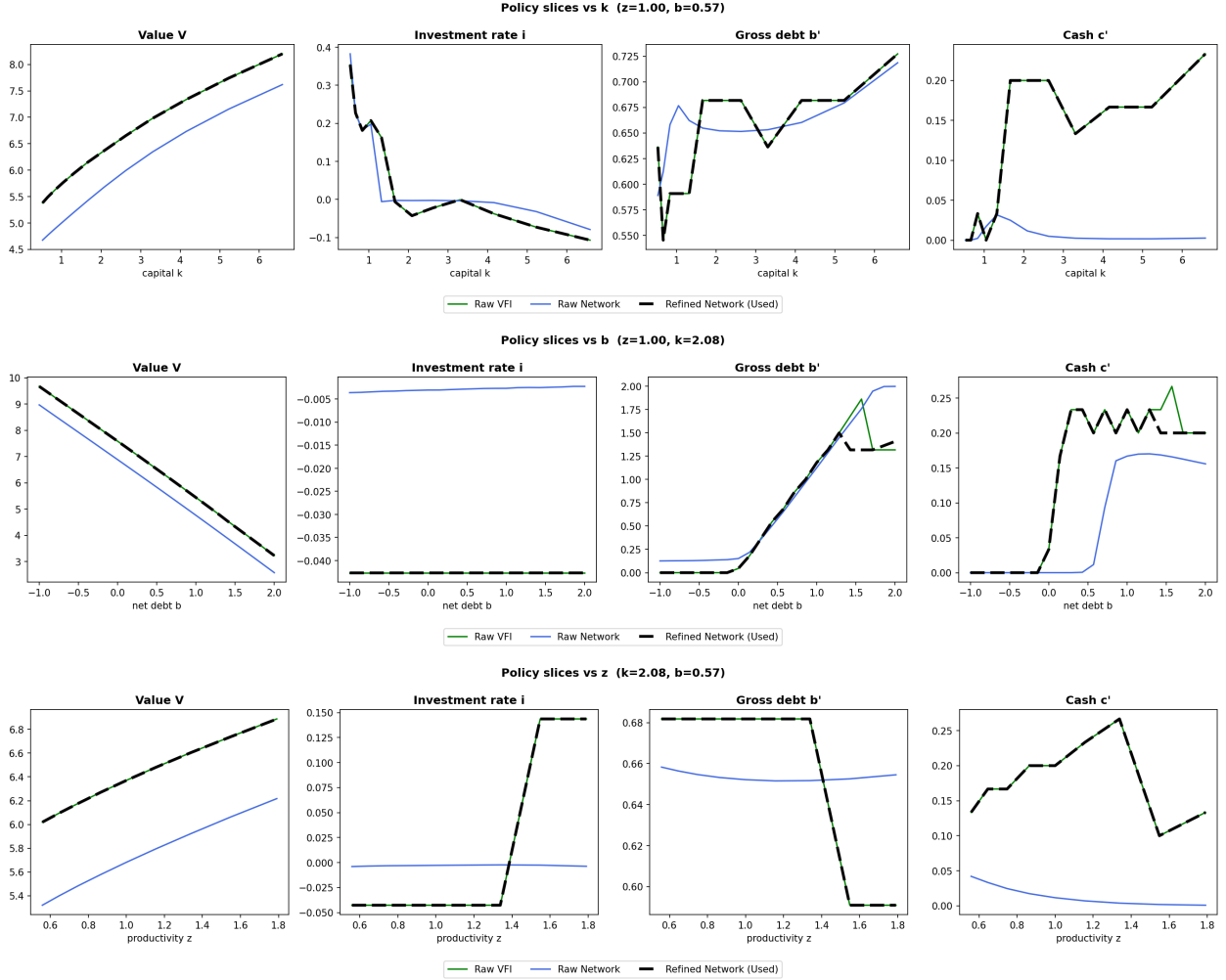


Figure 3.1: Value and policy function slices: refined network versus VFI benchmark.

Notes: Each row varies one state variable (capital k , debt b , productivity z) with the others held at reference values; columns are the value V , investment rate i , gross debt b' , and cash c' . The refined network (used downstream) overlays the grid VFI benchmark; the raw network is shown for comparison.

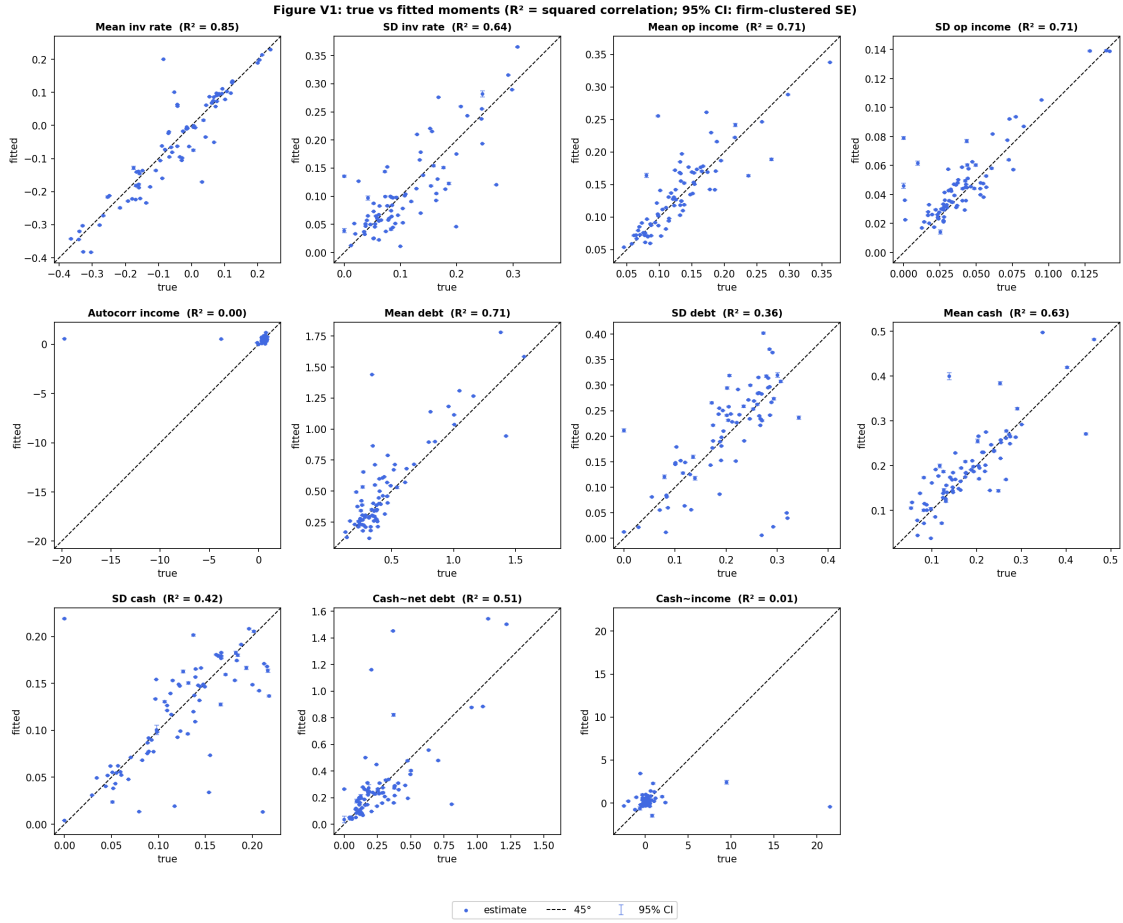


Figure 3.2: Moment recovery: fitted versus true moments across replications.

Notes: Each panel plots the fitted (model-implied) value of one of the 11 target moments against its true value across Monte Carlo replications; the dashed line is perfect recovery (45°) and R^2 in each panel title is the squared correlation. Per-moment values are in Table 3.1.

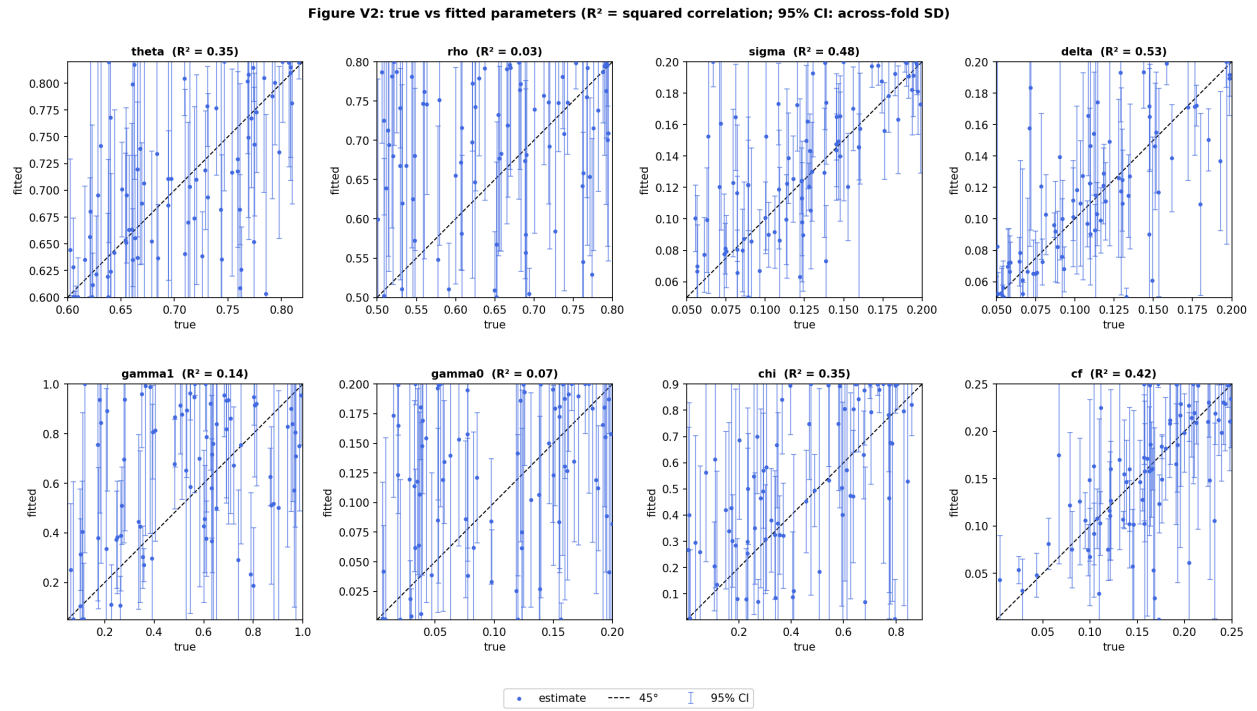


Figure 3.3: Parameter recovery: estimated versus true parameters across replications.

Notes: Each panel plots the estimate of one of the eight structural parameters against its true value, with 95% confidence intervals; the dashed line is perfect recovery and R^2 in each panel title is the squared correlation. Per-parameter values are in Table 3.2.

Table 3.1: Moment recovery summary.

Moment	True	Fitted	95% CI	R^2
Mean inv rate	-0.051	-0.056	[-0.057, -0.056]	0.85
SD inv rate	0.111	0.110	[0.109, 0.111]	0.64
Mean op income	0.131	0.137	[0.136, 0.137]	0.71
SD op income	0.041	0.047	[0.047, 0.048]	0.71
Autocorr income	0.251	0.567	[0.558, 0.577]	0.00
Mean debt	0.454	0.512	[0.510, 0.514]	0.71
SD debt	0.202	0.202	[0.201, 0.203]	0.36
Mean cash	0.183	0.195	[0.194, 0.196]	0.63
SD cash	0.123	0.117	[0.117, 0.118]	0.42
Cash~net debt	0.270	0.293	[0.290, 0.296]	0.51
Cash~income	0.514	0.231	[0.202, 0.260]	0.01

Notes: Mean across Monte Carlo replications of each of the 11 target moments: true value, fitted (model-implied) value, 95% confidence interval (firm-clustered standard errors), and R^2 (squared correlation between fitted and true). Corresponds to Figure 3.2.

Table 3.2: Parameter recovery summary.

Parameter	True	Estimate	RMSE	95% CI	R^2
θ (returns to scale)	0.703	0.703	0.062	[0.610, 0.796]	0.35
ρ (persistence)	0.650	0.693	0.127	[0.547, 0.839]	0.03
σ (shock SD)	0.125	0.135	0.035	[0.097, 0.173]	0.48
δ (depreciation)	0.111	0.113	0.032	[0.076, 0.151]	0.53
γ_1 (convex adj. cost)	0.525	0.636	0.330	[0.278, 0.994]	0.14
γ_0 (fixed adj. cost)	0.104	0.123	0.079	[0.023, 0.223]	0.07
χ (recovery rate)	0.454	0.532	0.252	[0.203, 0.860]	0.35
c_f (fixed op. cost)	0.157	0.153	0.052	[0.087, 0.220]	0.42

Notes: Across Monte Carlo replications for each of the eight structural parameters: true value, mean estimate, root-mean-squared error, 95% confidence interval (across-fold standard deviation), and R^2 (squared correlation between estimate and true). Corresponds to Figure 3.3.

Appendix: Interview QA

The interview questions and answers follow as a separately appended PDF.